



In-Database Learning with Sparse Tensors

Mahmoud Abo Khamis
RelationalAI, Inc

Hung Q. Ngo
RelationalAI, Inc

XuanLong Nguyen
University of Michigan

Dan Olteanu
University of Oxford

Maximilian Schleich
University of Oxford

ABSTRACT

In-database analytics is of great practical importance as it avoids the costly repeated loop data scientists have to deal with on a daily basis: select features, export the data, convert data format, train models using an external tool, reimport the parameters. It is also a fertile ground of theoretically fundamental and challenging problems at the intersection of relational and statistical data models.

This paper introduces a unified framework for training and evaluating a class of statistical learning models inside a relational database. This class includes ridge linear regression, polynomial regression, factorization machines, and principal component analysis. We show that, by synergizing key tools from relational database theory such as schema information, query structure, recent advances in query evaluation algorithms, and from linear algebra such as various tensor and matrix operations, one can formulate in-database learning problems and design efficient algorithms to solve them.

The algorithms and models proposed in the paper have already been implemented and deployed in retail-planning and forecasting applications, with significant performance benefits over out-of-database solutions that require the costly data-export loop.

KEYWORDS

In-database analytics; Functional aggregate queries; Functional dependencies; Model reparameterization; Tensors

ACM Reference Format:

Mahmoud Abo Khamis, Hung Q. Ngo, XuanLong Nguyen, Dan Olteanu, and Maximilian Schleich. 2018. In-Database Learning with Sparse Tensors. In *PODS'18: 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, June 10–15, 2018, Houston, TX, USA*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3196959.3196960>

1 INTRODUCTION

Although both disciplines of databases and statistics occupy foundational roles for the emerging field of data science, they are largely seen as complementary. Most fundamental contributions made by statisticians and machine learning researchers are abstracted away from the underlying infrastructure for data management. However, there is undoubtedly clear value in tight integration of statistics and

database models and techniques. A prime example of such a tight integration is provided by in-database analytics, which is receiving an increasing interest in both academia and industry [2, 34, 44]. This is motivated by the realization that in many practical cases data resides inside databases and bringing the analytics closer to the data saves non-trivial time usually spent on data import/export at the interface between database systems and statistical packages [30]. A complementary realization is that large chunks of statistical machine learning code can be expressed as relational queries and computed inside the database [19, 21, 35, 48]. In-database analytics problems naturally lend themselves to a systematic investigation using the toolbox of concepts and techniques developed by the database theorist, and by synergizing ideas from both relational and statistical data modeling. To solve optimization problems over relational data, one can exploit database schema information, functional dependencies, state-of-the-art query evaluation algorithms, and well-understood complexity analysis.

Our *conceptual contribution* in this paper is the introduction of a unified in-database framework for training and evaluating a class of statistical learning models. This class, commonly used in retail-planning and forecasting applications [11], includes ridge linear regression, polynomial regression, factorization machines, classification, and principal component analysis.

In such applications, the training dataset is the result of a feature extraction query over the database. Typical databases include weekly sales data, promotions, and product descriptions. A retailer would like to compute a parameterized model, which can predict, for instance, the additional demand generated for a given product due to promotion. As is prevalent in practical machine learning, the models are trained using a first-order optimization algorithm such as batch or stochastic gradient descent, in part because their convergence rates are dimension-free (for well-behaved objectives). This is a crucial property given the high-dimensionality of our problem as elaborated next.

The main *computational challenge* posed by in-database analytics is the large number of records and of features in the training dataset. There are two types of features: continuous (quantitative) such as price and revenue; and categorical (qualitative) such as colors, cities, and countries.¹ While continuous features allow for aggregation over their domains, categorical features cannot be aggregated together. To accommodate the latter, the state-of-the-art approach is to one-hot encode their active domain: each value in the active domain of an attribute is encoded by an indicator vector whose dimension is the size of the domain. For instance, the colors in the domain {red, green, blue} can be represented by indicator vectors [1, 0, 0] for red, [0, 1, 0] for green, and [0, 0, 1] for blue. The one-hot

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS'18, June 10–15, 2018, Houston, TX, USA

© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.

ACM ISBN 978-1-4503-4706-8/18/06...\$15.00

<https://doi.org/10.1145/3196959.3196960>

¹Most of the features we observed in datasets for retail applications are categorical.

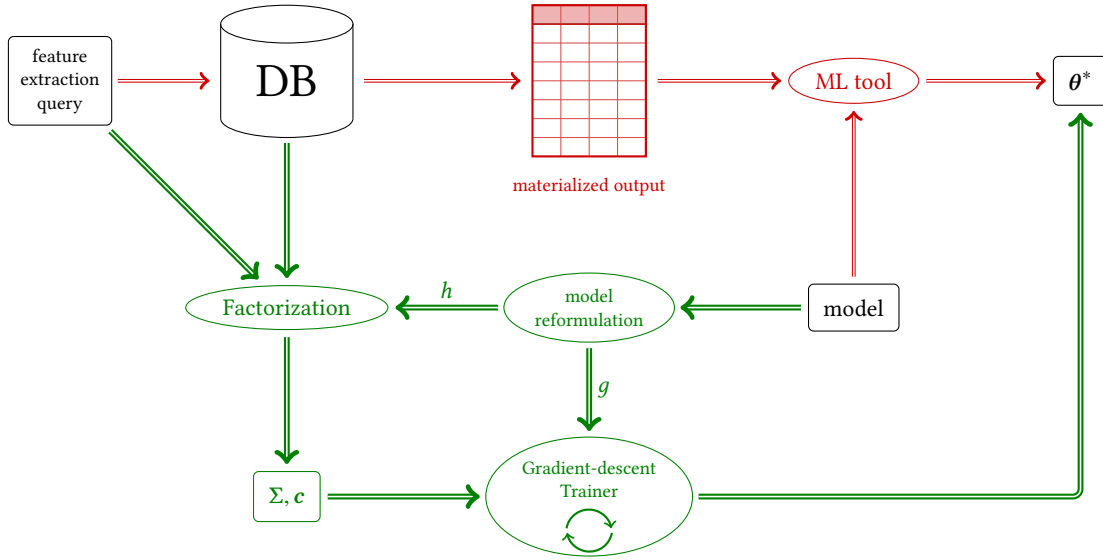


Figure 1: In-database vs. Out-of-database learning: High-level diagram. In **out-of-database** learning, a feature extraction query is issued to the database engine which computes, materializes and exports the output table. Depending on the chosen data model, the output table is then fed into an appropriate machine learning tool that learns and outputs the model parameters θ^* . Our **in-database** learning framework has a unified model formulation that subsumes many commonly-used models: Choosing a specific model reduces to choosing a feature map h and a function g of the model parameters. The feature map h , feature extraction query, and input tables are all fed into a factorization-based query engine, which computes a relatively small “digest” in the form of a matrix Σ and a vector c . The gradient-descent trainer only needs this digest and the function g to compute the model parameters θ^* , without ever going back to the input database.

encoding amounts to a relational representation of the training dataset with one new attribute per distinct category of each categorical feature and with wide tuples whose values are mostly 0. This entails huge redundancy due to the presence of the many 0 values. The one-hot encoding also blurs the usual database-theory distinction between schema and data, since the schema can become as large as the input database.

Closely related to the computational challenge is a *cultural challenge*: the feasibility of in-database analytics is often called into question. In terms of pure algorithmic performance, why would an in-database optimization algorithm be more efficient than an out-of-database optimization implementation, given the widely available plethora of tools and techniques for the latter?

Our answer to these challenges is that, for a large class of feature extraction queries, it is possible to train a model in time *sub-linear* in the output size of the feature extraction query! More concretely, our approach entails *three database-centric technical contributions*.

First, we exploit join dependencies and their factorization in the training dataset to asymptotically improve the per-iteration computation time of a gradient descent algorithm.

Second, we exploit functional dependencies present in the database to reduce the dimensionality of the underlying optimization problem by only optimizing for those parameters that functionally determine the others and by subsequently recovering the functionally determined parameters using their dependencies.

Third, we address the shortcomings of one-hot encoding by expressing the sum-product aggregates used to compute the gradient

and point evaluation as functional aggregate queries (FAQs) [8]. The aggregates over continuous features are expressed as FAQs without free variables and their computation yields scalar values. In contrast, aggregates over categorical features originating from a set S of database attributes are expressed as FAQs with free (i.e., group-by) variables S . The tuples in the result of such FAQs are combinations of categorical values that occur in the database. The ensemble of FAQs defining the gradient form a *sparse tensor representation and computation solution* with lower space and time complexity than solutions based on one-hot encoding. In particular, the complexity of our end-to-end solution can be arbitrarily smaller than that of materializing the result of the feature extraction query.

Figure 1 depicts schematically the workflows of our in-database approach and of existing mainstream out-of-database approaches for solving optimization problems.

Organization. The structure of the paper follows our contributions. Section 2 introduces our unified framework for in-database analytics. Section 3 introduces our sparse tensor representation and computation approach. Section 4 shows how to exploit functional dependencies to reduce the dimensionality for factorization machines and polynomial regression models.

Proofs of all theorems are in Appendix and the extended Technical Report of this paper [4]. The results presented in this paper form the foundation of an in-database analytics prototype. In experiments with real data, they show up to three orders of magnitude performance improvements over state-of-the-art competitors for polynomial regression models and factorization machines [5].

Related work. It has been recently acknowledged that database theory can effectively contribute to the arms race for in-database analytics [2]. Recent works highlight the potential of applying key database theory tools to this growing research of practical interest, e.g., the formal relational framework for classifier engineering [33] and in-database factorized learning of regression models with low data complexity [48].

Most related efforts in the database and distributed systems communities are on designing systems to support machine learning libraries or statistical packages on top of large-scale database architectures, e.g., MLLib [37] and DeepDist [38] on Spark [50], GLADE [45], TensorFlow [1], and SystemML [15, 31]. This approach relies on the expensive data export/import at the interface between the machine learning library process and the database system process: The feature extraction query is computed inside the database system, its result exported and imported into the data format of a machine learning library, where the model is learned. This approach is very expensive due to the import/export step and the unnecessary repetitions of data blocks in the query result as prescribed by the joins in the query. Our approach avoids these pitfalls. It differs from all these efforts in that it tightly integrates the analytics with the database query engine.

There are three lines of prior work closest to ours.

One line of work investigates the ability to express parts of analytical tasks within query languages. An important bulk of early work is on query languages with data mining, also called descriptive or backward-looking analytics, capabilities [17] and in-database data mining solutions, e.g., frequent itemsets [42] and association rule mining [10]. More recent work investigated how to (partly) express predictive (forward-looking) analytics, such as learning regression models and Naïve Bayes classification, together with the feature extraction query as a single optimized query with joins and sum-product aggregates [35, 48]. MADlib [30] casts analytics as user-defined aggregate functions (UDAFs) that can be used in SQL queries and executed inside PostgreSQL. These UDAFs remain black boxes for the underlying query engine, which has to compute the feature extraction query and delegate the UDAF computation on top of the query result to the MADlib's specialized code.

A second line of work exploits join dependencies for efficient in-database analytics. Join dependencies form the basis of the theory of (generalized) hypertree decompositions [25] and factorized databases [41], with applications such as inference in probabilistic graphical models, CSP, SAT, and databases. In databases, they have been originally used as a tractability yardstick for Boolean conjunctive queries [25] and more recently for the computation and result representation of queries with free variables [41], with group-by aggregates [8, 13], and with order-by clauses [13]. Our approach builds on earlier work that exploits join dependencies for learning linear regression models with continuous features [48]. Factorization machines [47] represent a regression model used for real-world analytics and that we investigate in this paper. In contrast to polynomial regression models, factorization machines factorize the space of model parameters to better capture data correlations. We further this idea by also factorizing the training dataset, which relies on join dependencies present in the data.

A third line of prior work uses functional dependencies (FDs) to avoid key-foreign key joins and reduce the number of features in

Naïve Bayes classification and feature selection [36]. We consider the effect of FDs on the reparameterization of regression models, where a non-trivial development is on the effect of FDs on the model (non-linear) regularization function.

Several state-of-the-art machine learning systems use a sparse representation of the input data to avoid redundancy introduced by one-hot encoding [20, 46]. In our setting, however, such systems require an additional data transformation step after the result of the feature extraction query is exported. This additional step is time consuming and makes the use of such systems very inefficient in many practical applications. In statistics and machine learning, there is a rich literature on learning with sparse and/or multilinear structures [29]. Such methods complement our framework and it would be of interest to leverage and adapt them to our setting.

Finally, there is a large collection of gradient-based methods proposed in the optimization literature. The description of our approach assumes batch gradient descent (BGD), though our insights are applicable to other methods, including Quasi-Newton algorithms. The main rationale for our choice is simplicity and good statistical properties. When combined with backtracking line search (as we do in this paper) or second-order gradient estimation (as in Quasi-Newton methods), BGD is guaranteed to converge to a minimum with linear asymptotic convergence rate. A naïve computation of the gradient requires a full pass over the data, which can be inefficient in large-scale analytics. A popular alternative is stochastic gradient descent (SGD), which estimates the gradient with a randomly selected mini-batch of training samples. The convergence of SGD, however, is noisy, requires careful setting of hyperparameters, and does not achieve the linear asymptotic convergence rate of BGD [16]. In our setting, the entire BGD execution can be arbitrarily faster than one SGD iteration over the result of the feature extraction query [5].

2 PROBLEM FORMULATION

The goal of this section is to present a general optimization formulation encompassing a range of machine learning tasks, and then to lay out a versatile mathematical representation suitable for the in-database treatment of these tasks.

Notational Convention. Bold face letters, e.g., $\mathbf{x}$, $\boldsymbol{\theta}$, $\mathbf{x}_i$, $\boldsymbol{\theta}_j$, denote vectors or matrices, and normal face letters, e.g., x_i , θ_j , $\theta_i^{(j)}$, denote scalars. For any positive integer n , $[n]$ denotes the set $\{1, \dots, n\}$. For any set S and positive integer k , $\binom{S}{k}$ denotes the collection of all k -subsets of S . We use the following matrix operations: $\otimes$ denotes the Kronecker/tensor product; $\circ$ the Hadamard product; $\star$ the Khatri-Rao product; and $\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product of two matrices, which reduces to the vector inner product when the matrices have one column each.

Let S be a finite set and Dom be any domain, then $\mathbf{a}_S = (a_j)_{j \in S} \in \text{Dom}^{|S|}$ is a *tuple* indexed by S , whose components are in Dom . If S and T are disjoint, and given tuples $\mathbf{a}_S$ and $\mathbf{a}_T$, then tuple $(\mathbf{a}_S, \mathbf{a}_T)$ is interpreted naturally as the tuple $\mathbf{a}_{S \cup T}$. The tuple $\mathbf{0}_S$ is the all-0 tuple indexed by S . If $S \subseteq G$, then the tuple $\mathbf{1}_{S|G}$ is the characteristic vector of the subset S , i.e., $\mathbf{1}_{S|G}(v) = 1$ if $v \in S$, and 0 if $v \in G - S$.

We make extensive use of basic concepts and results from matrix calculus summarized in Appendix A, where we also discuss a connection between tensor computation and the FAQ-framework [8].

Feature Extraction Query. We consider the setting where the training dataset D used as input to machine learning is the result of a natural join query Q , called *feature extraction query*, over a relational database I . Each tuple $(\mathbf{x}, y) \in D$ contains a scalar response (regressand) y and a tuple $\mathbf{x}$ encoding features (regressors).

We use standard notation for query hypergraphs. Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ denote the hypergraph of the query Q , where $\mathcal{V}$ is the set of variables occurring in Q and $\mathcal{E}$ is the set of hyperedges with one hyperedge per set of variables in a relation symbol R in the body of Q . We denote by $V \subseteq \mathcal{V}$ the subset of variables selected as features, and let $n = |V|$. The features in V corresponding to qualitative attributes are called *categorical*, while those corresponding to quantitative attributes are *continuous*. Let N be the size of the largest input relation R in Q .

Example 1. Consider the following natural join query Q that is a highly simplified version of a feature extraction query:

$Q(\text{sku}, \text{store}, \text{day}, \text{color}, \text{quarter}, \text{city}, \text{country}, \text{unitsSold})$
 $\leftarrow R_1(\text{sku}, \text{store}, \text{day}, \text{unitsSold}), R_2(\text{sku}, \text{color}),$
 $R_3(\text{day}, \text{quarter}), R_4(\text{store}, \text{city}), R_5(\text{city}, \text{country}).$

Relation R_1 records the number of units of a given sku (stock keeping unit) sold at a store on a particular day. The retailer is a global business, so it has stores in different cities and countries. One objective is to predict the number of blue units to be sold next year in the Fall quarter in Berlin. The response is the continuous variable *unitsSold*, $\mathcal{V}$ is the set of all variables, and $V = \mathcal{V} - \{\text{unitsSold}, \text{day}\}$, all of which are categorical.

2.1 Formulation with continuous features

Let $m \geq n$ be an integer. The feature map $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$ transforms the raw input vector $\mathbf{x} \in \mathbb{R}^n$ into an m -vector of “monomial features” $h(\mathbf{x}) = (h_j(\mathbf{x}))_{j \in [m]}$. Each component h_j is a multivariate *monomial* designed to capture the *interactions* among dimensions of input $\mathbf{x}$. In particular, we write $h_j(\mathbf{x}) := \prod_{i \in [n]} x_i^{a_j(i)}$, where degree $a_j(i)$ represents the level of participation of input dimension i in the j -th monomial feature. Let p be the number of parameters $\theta = (\theta_1, \dots, \theta_p) \in \mathbb{R}^p$, which produce the coefficients associated with features h via parameter map $g : \mathbb{R}^p \rightarrow \mathbb{R}^m$, $g(\theta) = (g_j(\theta))_{j \in [m]}$. Each component g_j is a multivariate *polynomial* of θ .

A large number of machine learning tasks learn a functional quantity of the form $\langle g(\theta), h(\mathbf{x}) \rangle$, where the parameters θ are obtained by solving $\min_{\theta} J(\theta)$ with

$$J(\theta) = \sum_{(\mathbf{x}, y) \in D} \mathcal{L}(\langle g(\theta), h(\mathbf{x}) \rangle, y) + \Omega(\theta). \quad (1)$$

$\mathcal{L}$ is a loss function, e.g., square loss, and Ω is a regularizer, e.g., ℓ_1 - or ℓ_2 -norm of θ . For square loss and ℓ_2 -regularization, $J(\theta)$ becomes:

$$J(\theta) = \frac{1}{2|D|} \sum_{(\mathbf{x}, y) \in D} (\langle g(\theta), h(\mathbf{x}) \rangle - y)^2 + \frac{\lambda}{2} \|\theta\|_2^2. \quad (2)$$

Example 2. The *ridge linear regression* (LR) model with response y and regressors $x_1, \dots, x_n$ has $p = n + 1$, parameters $\theta = (\theta_0, \dots, \theta_n)$. For convenience, we set $x_0 = 1$ corresponding to the bias parameter θ_0 . Then, $m = n + 1$, $g(\theta) = \theta$, and $h(\mathbf{x}) = \mathbf{x}$.

Example 3. The *degree- d polynomial regression* (PR^d) model with response y and regressors $x_0 = 1, x_1, \dots, x_n$ has $p = m = \binom{n+d}{d} = \sum_{i=0}^d \binom{n+i-1}{i}$ parameters $\theta = (\theta_a)$, where $\mathbf{a} = (a_1, \dots, a_n)$ is a tuple of non-negative integers such that $\|\mathbf{a}\|_1 \leq d$. In this case, $g(\theta) = \theta$, while the components of h are given by $h_a(\mathbf{x}) = \prod_{i=1}^n x_i^{a_i}$.

Example 4. The *degree-2 rank- r factorization machines* (FaMa_r^2) model with regressors $x_0 = 1, x_1, \dots, x_n$ and regressand y has parameters θ consisting of θ_i for $i \in \{0, \dots, n\}$ and $\theta_{ij}^{(\ell)}$ for $i \in [n]$ and $\ell \in [r]$. Training FaMa_r^2 corresponds to minimizing the following $J(\theta)$:

$$\frac{1}{2|D|} \sum_{(\mathbf{x}, y) \in D} \left(\sum_{i=0}^n \theta_i x_i + \sum_{\substack{\{i,j\} \in \binom{[n]}{2} \\ \ell \in [r]}} \theta_{ij}^{(\ell)} \theta_j^{(\ell)} x_i x_j - y \right)^2 + \frac{\lambda}{2} \|\theta\|_2^2.$$

This loss function follows Equation (2) with $p = 1 + n + rn$, $m = 1 + n + \binom{n}{2}$, and the parameter maps

$$h_S(\mathbf{x}) = \prod_{i \in S} x_i, \text{ for } S \subseteq [n], |S| \leq 2$$

$$g_S(\theta) = \begin{cases} \theta_0 & \text{when } |S| = 0 \\ \theta_i & \text{when } S = \{i\} \\ \sum_{\ell=1}^r \theta_{ij}^{(\ell)} \theta_j^{(\ell)} & \text{when } S = \{i, j\}. \end{cases}$$

Example 5. *Classification methods* such as support vector machines (SVM), logistic regression and Adaboost also fall under the same optimization framework, but with different choices of loss $\mathcal{L}$ and regularizer Ω . Typically, $\Omega(\theta) = \frac{\lambda}{2} \|\theta\|_2^2$. Restricting to binary class labels $y \in \{\pm 1\}$, the loss function $\mathcal{L}(\gamma, y)$, where $\gamma := \langle g(\theta), h(\mathbf{x}) \rangle$, takes the form $\mathcal{L}(\gamma, y) = \max\{1 - y\gamma, 0\}$ for SVM, $\mathcal{L}(\gamma, y) = \log(1 + e^{-y\gamma})$ for logistic regression and $\mathcal{L}(\gamma, y) = e^{-y\gamma}$ for Adaboost.

Example 6. Various unsupervised learning techniques can be expressed as iterative optimization procedures according to which each iteration is reduced to an optimization problem of the generic form given above. For example, the *Principal Component Analysis* (PCA) requires solving the following optimization problem to obtain a principal component direction

$$\max_{\|\theta\|=1} \theta^\top \Sigma \theta = \max_{\theta \in \mathbb{R}^p} \min_{\lambda \in \mathbb{R}} \theta^\top \Sigma \theta + \lambda(\|\theta\|^2 - 1),$$

where $\Sigma := \frac{1}{|D|} \sum_{\mathbf{x} \in D} \mathbf{x} \mathbf{x}^\top$ is the (empirical) correlation matrix of the given data. Although there is no response/class label y , within each iteration of the above iteration, for a fixed λ , there is a loss function $\mathcal{L}$ acting on feature vector $h(\mathbf{x})$ and parameter vector $g(\theta)$, along with a regularizer Ω . Specifically, we have $h(\mathbf{x}) = \Sigma \in \mathbb{R}^{p \times p}$, $g(\theta) = \theta \otimes \theta \in \mathbb{R}^{p \times p}$, $\mathcal{L} = \langle g(\theta), h(\mathbf{x}) \rangle_F$, where the Frobenius inner product is now employed. In addition, $\Omega(\theta) = \lambda(\|\theta\|^2 - 1)$.

2.2 Categorical features

The active domain of a categorical feature/variable consists of a set of possible values or categories. For example, vietnam, england, and usa are possible categories of the categorical feature country. Categorical features constitute the vast majority (up to 99%) of features we observed in our clients' machine learning applications.

It is common practice to one-hot encode categorical variables [28]. Whereas a continuous variable such as salary is mapped to a scalar value x_{salary} , a categorical variable such as country is mapped to an indicator vector $\mathbf{x}_{\text{country}}$ – a vector of binary values indicating the category that the variable takes on. For example, if the active domain of country consists of vietnam, england, and usa, then $\mathbf{x}_{\text{country}} = [x_{\text{vietnam}}, x_{\text{england}}, x_{\text{usa}}] \in \{0, 1\}^3$. If a tuple in the training dataset has country = “england”, then $\mathbf{x}_{\text{country}} = [0, 1, 0]$ for that tuple.

In general, the feature vector $\mathbf{x}$ in a tuple $(\mathbf{x}, y) \in D$ has the form $\mathbf{x} = (\mathbf{x}_c)_{c \in V}$, where each component $\mathbf{x}_c$ is an indicator vector if c is a categorical variable and a scalar otherwise. Similarly, each component of the parameter vector θ becomes a matrix (or a vector if the matrix has one column).

2.3 Tensor product representation

We accommodate both continuous and categorical features in our problem formulation (2) by replacing arithmetic product by tensor product in the component functions of the parameter map g and the feature map h . Specifically, monomials h_j now take the form

$$h_j(\mathbf{x}) = \bigotimes_{f \in V} \mathbf{x}_f^{a_j(f)} \quad (3)$$

with degree vector $\mathbf{a}_j = (a_j(f))_{f \in V} \in \mathbb{N}^n$. For each $j \in [m]$, the set $V_j := \{f \in V \mid a_j(f) > 0\}$ consists of features that participate in the interaction captured by the (hyper-) monomial h_j . Let $C \subseteq V$ denote the set of categorical variables and $C_j := C \cap V_j$ the subset of categorical variables in V_j . For $f \in C_j$, h_j represents $\prod_{f \in C_j} |\pi_f(D)|$ many monomials, one for each combination of the categories, where $\pi_f(D)$ denotes the projection of D onto variable f . Due to one-hot encoding, each element in the vector $\mathbf{x}_f$ for a categorical variable f is either 0 or 1, and $\mathbf{x}_f^{a_j(f)} = \mathbf{x}_f$ for $a_j(f) > 0$. Hence, h_j can be simplified as follows:

$$h_j(\mathbf{x}) = \prod_{f \in V_j - C_j} x_f^{a_j(f)} \cdot \bigotimes_{f \in C_j} \mathbf{x}_f. \quad (4)$$

Note that we use x_f instead of boldface $\mathbf{x}_f$ since each variable $f \in V_j - C_j$ is continuous.

Example 7. For illustration, consider a query that extracts tuples over schema (country, a , b , c , color) from the database, where country and color are categorical variables, while a, b, c are continuous variables. Moreover, there are two countries vietnam and england, and three colors red, green, and blue in the training dataset D . Consider three of the possible feature functions:

$$h_1(\mathbf{x}) = \mathbf{x}_{\text{country}} \otimes x_a^2 x_c \quad (5)$$

$$h_2(\mathbf{x}) = \mathbf{x}_{\text{country}} \otimes \mathbf{x}_{\text{color}} \otimes x_b \quad (6)$$

$$h_3(\mathbf{x}) = x_b x_c. \quad (7)$$

Under the one-hot encoding, the schema of the tuples becomes (vietnam, england, a, b, c , red, green, blue).

Equation (4) says that the functions h_1 and h_2 are actually encoding 8 functions:

$$\begin{aligned} h_{1, \text{vietnam}}(\mathbf{x}) &= x_{\text{vietnam}} x_a^2 x_c \\ h_{1, \text{england}}(\mathbf{x}) &= x_{\text{england}} x_a^2 x_c \\ h_{2, \text{vietnam, red}}(\mathbf{x}) &= x_{\text{vietnam}} x_{\text{red}} x_b \\ h_{2, \text{vietnam, green}}(\mathbf{x}) &= x_{\text{vietnam}} x_{\text{green}} x_b \\ h_{2, \text{vietnam, blue}}(\mathbf{x}) &= x_{\text{vietnam}} x_{\text{blue}} x_b \\ h_{2, \text{england, red}}(\mathbf{x}) &= x_{\text{england}} x_{\text{red}} x_b \\ h_{2, \text{england, green}}(\mathbf{x}) &= x_{\text{england}} x_{\text{green}} x_b \\ h_{2, \text{england, blue}}(\mathbf{x}) &= x_{\text{england}} x_{\text{blue}} x_b. \end{aligned}$$

We elaborate the tensor product representation for the considered learning models.

Example 8. In linear regression, parameter θ is a vector of vectors: $\theta = [\theta_0, \dots, \theta_n]$. Since our inner product is Frobenius, when computing $\langle \theta, \mathbf{x} \rangle$ we should be multiplying, for example, θ_{usa} with x_{usa} correspondingly.

Example 9. In polynomial regression, the parameter θ is a vector of tensors (i.e., high-dimensional matrices). Consider for instance the second order term $\theta_{ij} x_i x_j$. When both i and j are continuous, θ_{ij} is just a scalar. Now, suppose i is country and j is color. Then, the model has terms $\theta_{\text{vietnam, red}} x_{\text{vietnam}} x_{\text{red}}$, $\theta_{\text{usa, green}} x_{\text{usa}} x_{\text{green}}$, and so on. **All** these terms are captured by the Frobenius inner product $\langle \theta_{ij}, \mathbf{x}_i \otimes \mathbf{x}_j \rangle$. The component θ_{ij} is a matrix whose number of entries is the number of pairs (country, color) that appear together in some tuple in the training dataset. This number can be much less than the product of the numbers of countries and of colors in the input database.

Example 10. Consider the FaMa_r² model from Example (4), but now with categorical variables. From the previous examples, we already know how to interpret the linear part $\sum_{i=0}^n \theta_i x_i$ of the model when features are categorical. Consider a term in the quadratic part such as $\sum_{\ell \in [r]} \theta_i^{(\ell)} \theta_j^{(\ell)} x_i x_j$. When i and j are categorical, the term becomes $\left\langle \sum_{\ell \in [r]} \theta_i^{(\ell)} \otimes \theta_j^{(\ell)}, \mathbf{x}_i \otimes \mathbf{x}_j \right\rangle$.

3 FACTORIZED OPTIMIZATION

In this section we introduce our solution to learning statistical models for the setting of square loss function $J(\theta)$ and ℓ_2 -norm as in (2). We use a gradient-based optimization algorithm that employs the first-order gradient information to optimize the loss function $J(\theta)$. It repeatedly updates the parameters θ by some step size α in the direction of the gradient $\nabla J(\theta)$ until convergence. To guarantee convergence, it uses backtracking line search to ensure that α is sufficiently small to decrease the loss for each step. Each update step requires two computations: (1) *Point evaluation*: Given θ , compute the scalar $J(\theta)$; and (2) *Gradient computation*: Given θ , compute the vector $\nabla J(\theta)$. In particular, we use the batch gradient descent (BGD) algorithm with the Armijo line search condition and the Barzilai-Borwein step size adjustment [14, 22], as depicted in Algorithm 1. Quasi-Newton optimization algorithms (e.g., L-BFGS) and other common line search conditions are also applicable in our framework. We refer the reader to the excellent review article [24] for more details on fast implementations of the gradient-descent method.

Algorithm 1: BGD with Armijo line search.

```

 $\theta \leftarrow$  a random point;
while not converged yet do
   $\alpha \leftarrow$  next step size;
   $\mathbf{d} \leftarrow \nabla J(\theta)$ ;
  while  $J(\theta - \alpha \mathbf{d}) \geq J(\theta) - \frac{\alpha}{2} \|\mathbf{d}\|_2^2$  do
     $\alpha \leftarrow \alpha/2$  // line search;
   $\theta \leftarrow \theta - \alpha \mathbf{d}$ ;

```

3.1 Continuous features

To illustrate the main idea, we first consider the case without categorical features. We rewrite (2) to factor out the data-dependent part of the point evaluation and gradient computation. Recall that, for $j \in [m]$, h_j denotes the j th component function of the vector-valued function h , and h_j is a multivariate monomial in $\mathbf{x}$.

Theorem 3.1. *Let $J(\theta)$ be the function in (2). Define the matrix $\Sigma = (\sigma_{ij})_{i,j \in [m]}$, the vector $\mathbf{c} = (c_i)_{i \in [m]}$, and the scalar s_Y by*

$$\Sigma = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} h(\mathbf{x})h(\mathbf{x})^\top \quad (8)$$

$$\mathbf{c} = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} y \cdot h(\mathbf{x}) \quad (9)$$

$$s_Y = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} y^2. \quad (10)$$

Then,

$$J(\theta) = \frac{1}{2} g(\theta)^\top \Sigma g(\theta) - \langle g(\theta), \mathbf{c} \rangle + \frac{s_Y}{2} + \frac{\lambda}{2} \|\theta\|^2 \quad (11)$$

$$\nabla J(\theta) = \frac{\partial g(\theta)^\top}{\partial \theta} \Sigma g(\theta) - \frac{\partial g(\theta)^\top}{\partial \theta} \mathbf{c} + \lambda \theta. \quad (12)$$

Note that $\frac{\partial g(\theta)^\top}{\partial \theta}$ is a $p \times m$ matrix, and Σ is an $m \times m$ matrix. Statistically, Σ is related to the covariance matrix, $\mathbf{c}$ to the correlation between the response and the regressors, and s_Y to the empirical second moment of the response variable. Theorem 3.1 allows us to compute the two key steps of BGD *without* scanning through the data again, because the quantities $(\Sigma, \mathbf{c}, s_Y)$ can be computed efficiently in a preprocessing step *inside the database* as aggregates over the query Q . We shall elaborate on this point further in Section 3.3.

When g is the identity function, i.e., the model is linear, as is the case in PR and thus LR, (11) and (12) become particularly simple:

Corollary 3.2. *In a linear model (i.e., $g(\theta) = \theta$),*

$$J(\theta) = \frac{1}{2} \theta^\top \Sigma \theta - \langle \theta, \mathbf{c} \rangle + \frac{s_Y}{2} + \frac{\lambda}{2} \|\theta\|_2^2 \quad (13)$$

$$\nabla J(\theta) = \Sigma \theta + \lambda \theta - \mathbf{c}. \quad (14)$$

Let $\mathbf{d} = \nabla J(\theta)$. Then,

$$\nabla J(\theta - \alpha \mathbf{d}) = (1 - \alpha) \mathbf{d} - \alpha \Sigma \mathbf{d}. \quad (15)$$

The Armijo condition $J(\theta - \alpha \mathbf{d}) \geq J(\theta) - \frac{\alpha}{2} \|\mathbf{d}\|_2^2$ becomes:

$$\alpha \theta^\top \Sigma \mathbf{d} - \frac{\alpha^2}{2} \mathbf{d}^\top \Sigma \mathbf{d} - \alpha \langle \mathbf{c}, \mathbf{d} \rangle + \lambda \alpha \langle \theta, \mathbf{d} \rangle \leq \frac{\alpha}{2} (\lambda \alpha + 1) \|\mathbf{d}\|_2^2. \quad (16)$$

The significance of (16) is as follows. In a typical iteration of BGD, we have to backtrack a few times (say t times) for each value of α . If we were to recompute $J(\theta - \alpha \mathbf{d})$ using (13) each time, then the runtime of Armijo backtracking search is $O(tm^2)$, even after we have already computed $\mathbf{d}$ and $J(\theta)$. Now, using (16), we can compute in advance the following quantities (in this order): $\mathbf{d}$, $\|\theta\|_2^2$, $\Sigma \mathbf{d}$, $\langle \mathbf{c}, \mathbf{d} \rangle$, $\langle \theta, \mathbf{d} \rangle$, $\mathbf{d}^\top \Sigma \mathbf{d}$, $\theta^\top \Sigma \mathbf{d}$. Then, each check for inequality (16) can be done in $O(1)$ -time, for a total of $O(m^2 + t)$ -times. Once we have determined the step size α , (15) allows us to compute the next gradient (i.e., the next $\mathbf{d}$) in $O(m)$, because we have already computed $\Sigma \mathbf{d}$ for line search.

To implement BGD, we need to compute four quantities efficiently: the covariance matrix Σ in (8), the correlation vector $\mathbf{c}$ in (9), point evaluation in (11), and the gradient in (12). The covariance matrix and the correlation vector only have to be computed once in a pre-processing step. The gradient is computed at every iteration, which includes several point evaluations as we perform line search.² We do not need to compute the second moment s_Y because optimizing $J(\theta)$ is the same as optimizing $J(\theta) - s_Y$. Before describing how those four quantities can be computed efficiently, we discuss how we deal with categorical features.

3.2 Categorical features via sparse tensors

The more interesting, more common, and also considerably challenging situation is in the presence of categorical features. We next explain how we accommodate categorical features in the precomputation of Σ and $\mathbf{c}$.

Example 11. In Example 7, the matrix Σ is of size 8×8 instead of 3×3 after one-hot encoding. However, many of those entries are 0, for instance $(\forall (\mathbf{x}, y) \in D)$:

$$\begin{aligned} h_{1, \text{vietnam}}(\mathbf{x}) h_{1, \text{england}}(\mathbf{x}) &= 0 \\ h_{1, \text{england}}(\mathbf{x}) h_{2, \text{vietnam, blue}}(\mathbf{x}) &= 0 \\ h_{2, \text{vietnam, blue}}(\mathbf{x}) h_{2, \text{england, blue}}(\mathbf{x}) &= 0 \\ h_{2, \text{vietnam, blue}}(\mathbf{x}) h_{2, \text{vietnam, red}}(\mathbf{x}) &= 0. \end{aligned}$$

The reason is that the indicator variables x_{blue} and x_{england} act like selection clauses $x_{\text{color}} = \text{blue}$ and $x_{\text{country}} = \text{england}$. Thus, we can rewrite an entry σ_{ij} as an aggregate over a more selective query:

$$\sum_{(\mathbf{x}, y) \in D} h_{1, \text{vietnam}}(\mathbf{x}) h_{2, \text{vietnam, red}}(\mathbf{x}) = \sum_{\phi} x_a^2 x_c x_b,$$

$$\phi := ((\mathbf{x}, y) \in D \wedge x_{\text{color}} = \text{red} \wedge x_{\text{country}} = \text{vietnam}).$$

Extrapolating straightforwardly, if we were to write Σ down in the one-hot encoded feature space, then the entries σ_{ij} under one-hot encoding got unrolled into many entries. More concretely, σ_{ij} is in fact a tensor σ_{ij} of dimension $\prod_{f \in C_i} |\pi_f(D)| \times \prod_{f \in C_j} |\pi_f(D)|$, because

$$\sigma_{ij} = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} h_i(\mathbf{x}) h_j(\mathbf{x})^\top. \quad (17)$$

Similarly, each component c_j of $\mathbf{c}$ defined in (9) is a tensor $\mathbf{c}_j$ of dimension $\prod_{f \in C_j} |\pi_f(D)|$, because $h_j(\mathbf{x})$ is a tensor in the categorical case. The following follows immediately.

²In our implementation, each iteration typically involves 1-4 backtracking steps.

Theorem 3.3. *Theorem 3.1 remains valid even when some features are categorical.*

Note that the outer product in (17) specifies the matrix layout of σ_{ij} , and so Σ is a block matrix, each of whose blocks is σ_{ij} . Furthermore, if we were to layout the tensor σ_{ij} as a vector, we can also write it as

$$\sigma_{ij} = \frac{1}{|D|} \sum_{(x,y) \in D} h_i(x) \otimes h_j(y). \quad (18)$$

The previous example demonstrates that the dimensionalities of σ_{ij} and c_j can be very large. Fortunately, the tensors are very sparse, and a sparse representation of them can be computed with functional aggregate queries (in the FAQ-framework [8]) as shown in Proposition 3.4 below. We next illustrate the sparsity.

Example 12. Consider the query Q in Example 1, where the set of features is $\{\text{sku}, \text{store}, \text{day}, \text{color}, \text{quarter}, \text{city}, \text{country}\}$ and unitsSold is the response variable. In this query $n = 7$, and thus for a PR_2 model we have $m = 1 + 7 + \binom{8}{2} = 36$ parameters. Consider two indices i and j to the component functions of g and h , where $i = (\text{store}, \text{city})$ and $j = (\text{city})$. Suppose the query result states that the retailer has N_s stores in N_c countries. Then, the full dimensionality of the tensor σ_{ij} is $N_s \times N_c^2$, because by definition it was defined to be

$$\sigma_{ij} := \frac{1}{|D|} \sum_{(x,y) \in D} \underbrace{x_{\text{store}} \otimes x_{\text{city}}}_{h_i(x)} \otimes \underbrace{x_{\text{city}}}_{h_j(y)}. \quad (19)$$

Recall that x_{store} and x_{city} are both indicator vectors. The above tensor has the following straightforward interpretation: for every triple (s, c_1, c_2) , where s is a store and c_1 and c_2 are cities, this triple entry of the tensor counts the number of data points $(x, y) \in D$ for this particular combination of store and cities (divided by $1/|D|$). Most of these (s, c_1, c_2) -entries are 0. For example, if $c_1 \neq c_2$ then the count is zero. Thus, we can concentrate on computing entries of the form (s, c, c) :

```
SELECT s, c, count(*) FROM D GROUP BY s, c;
```

Better yet, since store functionally determines city, the number of entries in the query output is bounded by N_s . Using relations to represent sparse tensor results in a massive amount of space saving.

3.3 Efficient precomputation of Σ and c

We employ two orthogonal ideas to compute the quantities (8) and (9) efficiently. First, our FAQ [8] and FDB [48] frameworks are designed to compute such aggregates over feature extraction queries, which are wider than traditional OLAP queries. Let $|\sigma_{ij}|$ denote the *size* (i.e., number of tuples) of the sparse representation of the σ_{ij} tensor. Let $\text{faqw}(i, j)$ denote the FAQ-width of the FAQ-query that expresses the aggregate σ_{ij} over the feature extraction query Q^3 ; fhtw the fractional hypertree width of Q ; and ρ^* the fractional edge cover number of Q .⁴ Let I be the input database and $D = Q(I)$. Let N be the size of the largest input relation R in Q . Our precomputation time can be bounded as follows.

³We show in the proof of Proposition 3.4 how to express σ_{ij} and c_j as FAQ-queries.

⁴Due to space limitation, these width notions are defined in Appendix A.3.

Proposition 3.4. *The tensors σ_{ij} and c_j can be sparsely represented by FAQ-queries with group-by variables $C_i \cup C_j$ and C_j , respectively. They can be computed in time*

$$O\left(|V|^2 \cdot |\mathcal{E}| \cdot \sum_{i,j \in [m]} (N^{\text{faqw}(i,j)} + |\sigma_{ij}|) \cdot \log N\right).$$

In case all features in D are continuous, $C_j = \emptyset$ for all $j \in [m]$, and $\text{faqw}(i, j)$ is exactly the fractional hypertree width of Q [8]. Then, the overall runtime becomes $O(|V|^2 \cdot |\mathcal{E}| \cdot m^2 \cdot N^{\text{fhtw}} \cdot \log N)$. When some features are categorical, we can also bound the width $\text{faqw}(i, j)$ and tensor size, as stated by the next proposition.

Proposition 3.5. *Let $c = \max_{i,j} |C_i \cup C_j|$. Then, $\text{faqw}(i, j) \leq \text{fhtw} + c - 1$ and $|\sigma_{ij}| \leq \min\{|D|, N^c\}$, $\forall i, j \in [m]$. For any query Q with $\rho^* > \text{fhtw} + c - 1$, there are infinitely many database instances for which*

$$\lim_{N \rightarrow \infty} \frac{|D|}{\sum_{i,j \in [m]} (N^{\text{faqw}(i,j)} + |\sigma_{ij}|) \log N} = \infty. \quad (20)$$

Our precomputation step takes strictly sub-output-size runtime for infinitely many queries and database instances. If we were to compute σ_{ij} on a training dataset with categorical variables one-hot encoded, then the complexity would raise to $O(|V|^2 \cdot |\mathcal{E}| \cdot m^2 \cdot N^{\text{fhtw}+2d} \log N)$, where d is the degree of the model.

Second, we exploit the observation that in the computation of Σ many *distinct* tensors σ_{ij} have *identical* sparse representations. For instance, the tensor σ_{ij} from Example 12 corresponding to $i = (\text{store}, \text{city})$ and $j = (\text{city})$ has the same sparse representation as any of the following tensors: $(i, j) \in \{((\text{city}, \text{city}), \text{store}), ((\text{store}, \text{store}), \text{city}), ((\text{store}, \text{city}), \text{store}), \dots\}$. There are 12 tensors sharing this particular sparse representation. This is because store and city are categorical features and taking any power of the binary values in their indicator vectors does not change these values. Furthermore, any of the two features can be in i and/or j .

3.4 Point evaluation and gradient computation

To compute point evaluation and the gradient efficiently, we also introduce two ideas. First, we employ a sparse representation of tensors in the *parameter space*. We need to evaluate the component functions of g , which are polynomial. In the FaMa_r² example, for instance, we evaluate expressions of the form

$$g_{\text{store}, \text{city}}(\theta) = \sum_{\ell=1}^r \theta_{\text{store}}^{(\ell)} \otimes \theta_{\text{city}}^{(\ell)}. \quad (21)$$

The result is a 2-way tensor whose CP-decomposition (a sum of rank-1 tensors) is already given by (21)! There is no point in materializing the result of $g_{\text{store}, \text{city}}(\theta)$ and we instead keep it as is. Assuming N_c distinct cities and N_s distinct stores in the training dataset D , if we were to materialize the tensor, then we would end up with an $\Omega(N_c N_s)$ -sized result for absolutely no gain in computational and space complexity, while the space complexity of the CP-decomposition is only $O(N_c + N_s)$. This is a prime example of factorization of the parameter space.

Second, we explain how to evaluate (11) and (12) with our sparse tensor representations. There are two aspects of our solution worth

spelling out: (1) how to multiply two tensors, e.g., σ_{ij} and $g_j(\theta)$, and (2) how to exploit that some tensors have the same representation to speed up the point evaluation and gradient computation.

To answer question (1), we need to know the intrinsic dimension of the tensor σ_{ij} . In order to compute $\Sigma g(\theta)$ in Example 12, we need to multiply σ_{ij} with $g_j(\theta)$ for $i = (\text{store}, \text{city})$ and $j = (\text{city})$. In a linear model, $g_j(\theta) = \theta_j = \theta_{\text{city}}$. In this case, when computing $\sigma_{ij}\theta_{\text{city}}$ we marginalize away one city dimension of the tensor, while keeping the other two dimensions store, city. This is captured by the following query:

```
SELECT store, city, sum( $\sigma_{i,j}.\text{val} * \theta_j.\text{val}$ )
FROM  $\sigma_{i,j}, \theta_j$  WHERE  $\sigma_{i,j}.\text{city} = \theta_j.\text{city}$ 
GROUP BY store, city;
```

where the tensors $\sigma_{i,j}$ and θ_j map (store, city) and respectively (city) to aggregate values. In words, $\sigma_{ij}g_j(\theta)$ is computed by a group-by aggregate query where the group-by variables are precisely the variables in C_i .

For the second question, we use the CP-decomposition of the parameter space as discussed earlier. Suppose now we are looking at the σ_{ij} tensor where $i = (\text{city})$ and $j = (\text{store}, \text{city})$. Note that this tensor has the identical representation as the above tensor, but it is a *different* tensor. In a FaMa_r² model, we would want to multiply this tensor with the component function $g_j(\theta)$ defined in (21) above. We do so by multiplying it with each of the terms $\theta_{\text{store}}^{(\ell)} \otimes \theta_{\text{city}}^{(\ell)}$, one by one for $\ell = 1, \dots, r$, and then add up the result. Multiplying the tensor σ_{ij} with the first term $\theta_{\text{store}}^{(1)} \otimes \theta_{\text{city}}^{(1)}$ corresponds precisely to the following query:

```
SELECT city, sum( $\sigma_{i,j}.\text{val} * \theta_{\text{store}}^{(1)}.\text{val} * \theta_{\text{city}}^{(1)}.\text{val}$ )
FROM  $\sigma_{i,j}, \theta_{\text{store}}^{(1)}, \theta_{\text{city}}^{(1)}$ 
WHERE  $\sigma_{i,j}.\text{city} = \theta_{\text{city}}^{(1)}.\text{city}$  AND
       $\sigma_{i,j}.\text{store} = \theta_{\text{store}}^{(1)}.\text{store}$ 
GROUP BY city;
```

where the tensors $\sigma_{i,j}$, $\theta_{\text{city}}^{(1)}$, and $\theta_{\text{store}}^{(1)}$ map (store, city), (city), and respectively (store) to aggregate values.

Finally, to answer question (2), note that for the same column j (i.e., the same component function $g_j(\theta)$), there can be multiple tensors σ_{ij} which have identical sparse representations. (This holds especially in models of degree > 1 .) In such cases, we have queries with identical from-where blocks but different select-group-by clauses, because the tensors have different group-by variables. Nevertheless, all such queries can share computation as we can compute the from-where clause once for all of them and then scan this result to compute each specific tensor. This analysis gives rise to the following straightforward (and conservative) estimates.

For each $j \in [m]$, let d_j denote the degree and t_j denote the number of terms in the polynomial g_j (a component function of g). Recall that p is the number of parameters.

Proposition 3.6. *Point evaluation (11) and gradient computation (12) can be computed in time $O(\sum_{i,j \in [m]} t_i t_j d_i d_j |\sigma_{ij}|)$, and respectively $O(p \sum_{i,j \in [m]} t_i t_j d_i d_j |\sigma_{ij}|)$.*

In the PR^d model, the point evaluation and gradient computation times are $O(d^2 \sum_{i,j \in [m]} |\sigma_{ij}|)$ and $O(n^d \sum_{i,j \in [m]} |\sigma_{ij}|)$, respectively. In the FaMa_r^d model, these times are $O(r^2 d^2 \sum_{i,j \in [m]} |\sigma_{ij}|)$ and $O(nr^3 d^2 \sum_{i,j \in [m]} |\sigma_{ij}|)$, respectively.

Overall, there are a couple of remarkable facts regarding the overall runtime of our approach. Without loss of generality, suppose the number of iterations of BGD is bounded. (This bound is typically dimension-free, dependent on the Lipschitz constant of f .) Then, from Proposition 3.5, there are infinitely many queries for which the overall runtime of BGD is unboundedly better than the output size. First, our approach is faster than even the data-export step of the “use an external tool to train model” approach. Second, it is often well-agreed upon that SGD is “faster” than BGD. However, a single iteration of SGD requires iterating through all data tuples, which takes time at least the output size. In particular, by training the model using BGD in the factorized form, BGD can be unboundedly faster than a single iteration of SGD.

4 FD-AWARE OPTIMIZATION

In this section, we show how to exploit functional dependencies among variables to reduce the dimensionality of the optimization problem by eliminating functionally determined variables and re-parameterizing the model. We compute the quantities (Σ, c) on the subset of features that are not functionally determined, and then solve the lower-dimensional optimization problem. Finally, we recover the parameters in the original space in closed form. Exploiting functional dependencies drastically reduces the computation time for (Σ, c) and the gradient.

4.1 Introduction to the main ideas

Consider a query Q with categorical variables country and city. For simplicity, assume that there are only two countries “vietnam” and “england”, and 5 cities “saigon”, “hanoi”, “oxford”, “leeds”, and “bristol”. Under one-hot encoding, the corresponding features are encoded as indicators $x_{\text{vietnam}}, x_{\text{england}}, x_{\text{saigon}}, x_{\text{hanoi}}, x_{\text{oxford}}, x_{\text{leeds}}, x_{\text{bristol}}$. Since $\text{city} \rightarrow \text{country}$ is a functional dependency (FD), for a given tuple x in the training dataset, the following hold:

$$x_{\text{vietnam}} = x_{\text{saigon}} + x_{\text{hanoi}} \quad (22)$$

$$x_{\text{england}} = x_{\text{oxford}} + x_{\text{leeds}} + x_{\text{bristol}} \quad (23)$$

The first identity states that if a tuple has “vietnam” as the value for country ($x_{\text{vietnam}} = 1$), then its value for city can only be either “saigon” or “hanoi”, i.e., $[x_{\text{saigon}}, x_{\text{hanoi}}]$ is either $[1, 0]$ or $[0, 1]$, respectively. The second identity is explained similarly.

How do we express the identities such as (22) and (23) in a formal manner in terms of the input vectors x_{city} and x_{country} ? We can extract in a preprocessing step from the database a relation of the form $R(\text{city}, \text{country})$ with city as primary key. Let N_{city} and N_{country} be the number of cities and countries, respectively. The predicate $R(\text{city}, \text{country})$ is the sparse representation of a matrix R of size $N_{\text{country}} \times N_{\text{city}}$, such that if x_{city} is an indicator vector for saigon, then Rx_{city} is an indicator for vietnam. In this language, the above identities are written as $x_{\text{country}} = Rx_{\text{city}}$. For example,

in the above particular example $N_{\text{city}} = 5$, $N_{\text{country}} = 2$, and

	saigon	hanoi	oxford	leeds	bristol	
$\mathbf{R} =$	1	1	0	0	0	vietnam
	0	0	1	1	1	england

This relationship suggests a natural idea: replace any occurrence of statistics $\mathbf{x}_{\text{country}}$ by its functionally determining quantity $\mathbf{x}_{\text{city}}$. Since these quantities are present only in the loss function $\mathcal{L}$ via inner products $\langle g(\mathbf{x}), h(\boldsymbol{\theta}) \rangle$, such replacements result in a (typically) linear reparameterization of the loss. What happens next is less obvious, due to the presence of the nonlinear penalty function Ω . Depending on the specific structure of FDs and the choice of Ω , many parameters associated with redundant statistics, which do not affect the loss $\mathcal{L}$, can be optimized out directly with respect to the transformed Ω penalty.

The remainder of this subsection is a gentle introduction of our idea in the presence of *one* simple FD in the LR model. Consider a query Q in which city and country are two of the categorical features and functionally determine one another via a matrix $\mathbf{R}$ such that $\mathbf{R}\mathbf{x}_{\text{city}} = \mathbf{x}_{\text{country}}$ for all $\mathbf{x} = (\dots, \mathbf{x}_{\text{city}}, \mathbf{x}_{\text{country}}, \dots) \in D$. We exploit this fact to “eliminate” $\mathbf{x}_{\text{country}}$ as follows.

$$\begin{aligned}
\langle g(\boldsymbol{\theta}), h(\mathbf{x}) \rangle &= \langle \boldsymbol{\theta}, \mathbf{x} \rangle \\
&= \sum_{j \notin \{\text{city}, \text{country}\}} \langle \boldsymbol{\theta}_j, \mathbf{x}_j \rangle + \langle \boldsymbol{\theta}_{\text{city}}, \mathbf{x}_{\text{city}} \rangle + \langle \boldsymbol{\theta}_{\text{country}}, \mathbf{x}_{\text{country}} \rangle \\
&= \sum_{j \notin \{\text{city}, \text{country}\}} \langle \boldsymbol{\theta}_j, \mathbf{x}_j \rangle + \langle \boldsymbol{\theta}_{\text{city}}, \mathbf{x}_{\text{city}} \rangle + \langle \boldsymbol{\theta}_{\text{country}}, \mathbf{R}\mathbf{x}_{\text{city}} \rangle \\
&= \sum_{j \notin \{\text{city}, \text{country}\}} \langle \boldsymbol{\theta}_j, \mathbf{x}_j \rangle + \underbrace{\left\langle \boldsymbol{\theta}_{\text{city}} + \mathbf{R}^T \boldsymbol{\theta}_{\text{country}}, \mathbf{x}_{\text{city}} \right\rangle}_{\boldsymbol{\gamma}_{\text{city}}}.
\end{aligned}$$

Reparameterize the model by defining $\boldsymbol{\gamma} = (\boldsymbol{\gamma}_j)_{j \in V - \{\text{country}\}}$, and two functions $\bar{g} : \mathbb{R}^{n-1} \rightarrow \mathbb{R}^{n-1}$, $\bar{h} : \mathbb{R}^n \rightarrow \mathbb{R}^{n-1}$:

$$\boldsymbol{\gamma}_j = \begin{cases} \boldsymbol{\theta}_j & j \neq \text{city} \\ \boldsymbol{\theta}_{\text{city}} + \mathbf{R}^T \boldsymbol{\theta}_{\text{country}} & j = \text{city}. \end{cases} \quad (24)$$

$$\bar{g}(\boldsymbol{\gamma}) = \boldsymbol{\gamma} \quad (25)$$

$$\bar{h}_j(\mathbf{x}) = \mathbf{x}_j, \quad j \neq \text{city}. \quad (26)$$

(There is no $\boldsymbol{\gamma}_{\text{country}}$.) Reparameterize $J(\boldsymbol{\theta})$ by

$$\begin{aligned}
J(\boldsymbol{\theta}) &= \frac{1}{2|D|} \sum_{(\mathbf{x}, y) \in D} (\langle g(\boldsymbol{\theta}), h(\mathbf{x}) \rangle - y)^2 + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 \\
&= \frac{1}{2|D|} \sum_{(\mathbf{x}, y) \in D} (\langle \bar{g}(\boldsymbol{\gamma}), \bar{h}(\mathbf{x}) \rangle - y)^2 + \\
&\quad \frac{\lambda}{2} \left(\sum_{j \neq \text{city}} \|\boldsymbol{\gamma}_j\|_2^2 + \|\boldsymbol{\gamma}_{\text{city}} - \mathbf{R}^T \boldsymbol{\theta}_{\text{country}}\|_2^2 + \|\boldsymbol{\theta}_{\text{country}}\|_2^2 \right).
\end{aligned}$$

Note how $\boldsymbol{\theta}_{\text{country}}$ has disappeared from the loss term, but it still remains in the penalty term. We now “optimize out” $\boldsymbol{\theta}_{\text{country}}$ by observing that

$$\frac{1}{\lambda} \frac{\partial J}{\partial \boldsymbol{\theta}_{\text{country}}} = \mathbf{R}(\mathbf{R}^T \boldsymbol{\theta}_{\text{country}} - \boldsymbol{\gamma}_{\text{city}}) + \boldsymbol{\theta}_{\text{country}} \quad (27)$$

By setting (27) to 0 we obtain $\boldsymbol{\theta}_{\text{country}}$ in terms of $\boldsymbol{\gamma}_{\text{city}}$: $\boldsymbol{\theta}_{\text{country}} = (\mathbf{I}_{\text{country}} + \mathbf{R}\mathbf{R}^T)^{-1} \mathbf{R}\boldsymbol{\gamma}_{\text{city}}$ and equivalently as $\mathbf{R}(\mathbf{I}_{\text{city}} + \mathbf{R}^T \mathbf{R})^{-1} \boldsymbol{\gamma}_{\text{city}}$, where $\mathbf{I}_{\text{country}}$ is the order- N_{country} identity matrix and similarly for $\mathbf{I}_{\text{city}}$. (See [4].) J can thus be expressed completely in terms of $\boldsymbol{\gamma}$, and its gradient with respect to $\boldsymbol{\gamma}$ is also available:

$$\begin{aligned}
J(\boldsymbol{\theta}) &= \frac{1}{2|D|} \sum_{(\mathbf{x}, y) \in D} (\langle \bar{g}(\boldsymbol{\gamma}), \bar{h}(\mathbf{x}) \rangle - y)^2 + \\
&\quad \frac{\lambda}{2} \left(\sum_{j \neq \text{city}} \|\boldsymbol{\gamma}_j\|_2^2 + \langle (\mathbf{I}_{\text{city}} + \mathbf{R}^T \mathbf{R})^{-1} \boldsymbol{\gamma}_{\text{city}}, \boldsymbol{\gamma}_{\text{city}} \rangle \right), \\
\frac{1}{2} \frac{\partial \|\boldsymbol{\theta}\|_2^2}{\partial \boldsymbol{\gamma}_j} &= \begin{cases} \boldsymbol{\gamma}_j & j \neq \text{city} \\ (\mathbf{I}_{\text{city}} + \mathbf{R}^T \mathbf{R})^{-1} \boldsymbol{\gamma}_{\text{city}} & j = \text{city}. \end{cases}
\end{aligned}$$

The gradient of the loss term is computed using the matrix $\bar{\Sigma}$ and the vector $\bar{\mathbf{c}}$ with respect to the pair $(\bar{g}, \bar{h})$ of reduced dimensionality. The matrix $(\mathbf{I}_{\text{city}} + \mathbf{R}^T \mathbf{R})$ is a rank- N_{country} update to the identity matrix $\mathbf{I}_{\text{city}}$, strictly positive definite and thus invertible. The inverse can be obtained using database aggregate queries; for numerical stability, one may compute its Cholesky decomposition which can also be expressed by aggregate queries. These “linear algebra via aggregate queries” computations are possible because our matrices admit a database interpretation, cf. Section 4.5.

4.2 Functional dependencies (FDs)

Composite FDs lead to more complex identities. For instance, the FD (guest, hotel, date) $\rightarrow$ room leads to the identity $x_{\text{room}} = \sum x_{\text{guest}} x_{\text{hotel}} x_{\text{date}}$. Let R be a relation on attributes guest, hotel, date, and room, encoding this dependency, i.e., R has a compound key (guest, hotel, date). Then, corresponding to R there is a matrix $\mathbf{R}$ of dimension $N_{\text{room}} \times N_{\text{guest}} \cdot N_{\text{hotel}} \cdot N_{\text{date}}$ for which $\mathbf{x}_{\text{room}} = \mathbf{R}(\mathbf{x}_{\text{guest}} \otimes \mathbf{x}_{\text{hotel}} \otimes \mathbf{x}_{\text{date}})$. Our results can be extended to the case of composite FDs, yet with a great notational burden; for the sake of clarity, we only state the results for *simple* FDs.

Definition 1. An FD is *simple* if its left-hand side is one variable.

Let a query Q in which there are k disjoint groups $G_1, \dots, G_k$ of features, among other features. The i th group is $G_i = \{f_i\} \cup S_i$, where f_i is a feature, S_i a set of features, and $f_i \rightarrow S_i$ is an FD. We shall refer to these as *groups of simple FDs*.

Example 13. In a typical feature extraction query for retailer customers, we have $k = 3$ groups (in addition to other features): the first group contains week $\rightarrow$ month $\rightarrow$ quarter $\rightarrow$ year, and thus $f_1 = \text{week}$ and $S_1 = \{\text{month, quarter, year}\}$. In the second group, $f_2 = \text{sku}$ and $S_2 = \{\text{type, color, size, ...}\}$ (a rather large group). In the third group $f_3 = \text{store}$ and $S_3 = \{\text{city, country, region, continent}\}$.

For each feature $c \in S_i$, let $\mathbf{R}_c$ denote the matrix for which $\mathbf{x}_c = \mathbf{R}_c \mathbf{x}_{f_i}$. For the sake of brevity, we also define a matrix $\mathbf{R}_{f_i} = \mathbf{I}_{f_i}$ (the identity matrix of dimension equal to the active domain size of attribute f_i), so the equality $\mathbf{R}_c \mathbf{x}_{f_i} = \mathbf{x}_c$ holds for every $c \in G_i$.

The linear relationship holds even if the variables are not categorical. For example, consider the FD sku $\rightarrow$ price (assuming every

stock-keeping unit has a fixed sale-price). The relationship is modeled with a $1 \times N_{\text{sku}}$ matrix $\mathbf{R}$, where the entry corresponding to a sku is its price. Then, $\mathbf{R}\mathbf{x}_{\text{sku}} = x_{\text{price}}$ for any indicator vector $\mathbf{x}_{\text{sku}}$.

Definition 2 (FD-reduced pairs of functions). Given a pair of functions g and h in our problem setting. Recall that C_j 's are defined in Section 2.3, while S_k 's are given in Definition 1. Define

$$K := \{j \in [m] \mid C_j \cap (S_1 \cup \dots \cup S_k) \neq \emptyset\}$$

(In words, K is the set of component functions of h containing at least one functionally determined variable.)

The group of simple FDs induces an *FD-reduced pair* of functions $\bar{g} : \mathbb{R}^{p-|K|} \rightarrow \mathbb{R}^{m-|K|}$ and $\bar{h} : \mathbb{R}^n \rightarrow \mathbb{R}^{m-|K|}$ as follows: The component functions of $\bar{h}$ are obtained from the component functions of h by removing all component functions h_j for $j \in K$. Similarly, $\bar{g}$ is obtained from g by removing all component functions g_j for which $j \in K$. Naturally, define the covariance matrix $\bar{\Sigma}$ and the correlation vector $\bar{c}$ as in (8) and (9), but with respect to $\bar{h}$.

We next generalize the above technique to speedup the training of PR^d and FaMa under an arbitrary collection of simple FDs.

4.3 Polynomial regression under FDs

Recall the PR^d -model formulated in Example 3. Consider the set A_V of all tuples $\mathbf{a}_V = (a_w)_{w \in V} \in \mathbb{N}^V$ of non-negative integers such that $\|\mathbf{a}_V\|_1 \leq d$. For any $(\mathbf{x}, y) \in D$ and $\mathbf{a} \in A_V$, define $\mathbf{x}^{\otimes \mathbf{a}} := \bigotimes_{v \in V} \mathbf{x}_v^{\otimes a_v}$. In the PR^d model we have $\theta = (\theta_{\mathbf{a}})_{\|\mathbf{a}\|_1 \leq d}$, $g(\theta) = \theta$, and $h_{\mathbf{a}}(\mathbf{x}) = \mathbf{x}^{\otimes \mathbf{a}}$. If a feature, say $v \in V$, is non-categorical, then $\mathbf{x}_v^{\otimes a_v} = x_v^{a_v}$. If we knew $x_v \in \{0, 1\}$, then $x_v^{a_v} = x_v$ and thus there is no need to have terms for which $a_v > 1$. A similar situation occurs when v is a categorical variable. To see this, let us consider a simple query where $V = \{b, c, w, t\}$, and all four variables are categorical. Suppose the PR^d model has a term corresponding to $\mathbf{a} = (a_b, a_c, a_w, a_t) = (0, 2, 0, 1)$. The term of $\langle \theta, h(\mathbf{x}) \rangle$ indexed by tuple $\mathbf{a}$ is of the form

$$\langle \theta_{\mathbf{a}}, \mathbf{x}_c^{\otimes 2} \otimes \mathbf{x}_t \rangle = \langle \theta_{\mathbf{a}}, \mathbf{x}_c \otimes \mathbf{x}_c \otimes \mathbf{x}_t \rangle.$$

For the dimensionality to match up, $\theta_{\mathbf{a}}$ is a 3rd-order tensor, say indexed by (i, j, k) . The above expression can be simplified as

$$\begin{aligned} \sum_i \sum_j \sum_k \theta_{\mathbf{a}}(i, j, k) \cdot \mathbf{x}_c(i) \cdot \mathbf{x}_c(j) \cdot \mathbf{x}_t(k) \\ = \sum_j \sum_k \theta_{\mathbf{a}}(j, j, k) \mathbf{x}_c(j) \mathbf{x}_t(k), \end{aligned}$$

where the equality holds due to the fact that $\mathbf{x}_c(j)$ is idempotent. In particular, we only need the entries indexed by (j, j, k) of $\theta_{\mathbf{a}}$. Equivalently, we write:

$$\langle \theta_{\mathbf{a}}, \mathbf{x}_c \otimes \mathbf{x}_c \otimes \mathbf{x}_t \rangle = \langle (\mathbf{I}_c \star \mathbf{I}_c)^\top \otimes \mathbf{I}_t, \theta_{\mathbf{a}}, \mathbf{x}_c \otimes \mathbf{x}_t \rangle.$$

Multiplying on the left by the matrix $(\mathbf{I}_c \star \mathbf{I}_c)^\top \otimes \mathbf{I}_t$ has precisely the same effect as selecting out only entries $\theta_{\mathbf{a}}(j, j, k)$ from the tensor $\theta_{\mathbf{a}}$. More generally, in the PR^d model we can assume that all the indices $\mathbf{a}_V = (a_v)_{v \in V}$ satisfy the condition that $a_v \in \{0, 1\}$ whenever v is categorical. (This is in addition to the degree requirement that $\|\mathbf{a}_V\|_1 \leq d$.)

Given k groups of FDs represented by $G_1, \dots, G_k$, let $G = \bigcup_{i=1}^k G_i$, $S = \bigcup_{i=1}^k S_i$, $\bar{G} = V - G$, $\bar{S} = V - S$, and $F = \{f_1, \dots, f_k\}$. For every non-empty subset $T \subseteq [k]$, define $F_T := \{f_i \mid i \in T\}$. Given a natural number $q < d$, and a non-empty set $T \subseteq [k]$ with size $|T| \leq d - q$, define the collection

$$\mathcal{U}(T, q) := \{U \mid U \subseteq G \wedge U \cap G_i \neq \emptyset, \forall i \in T \wedge U \cap G_i = \emptyset, \forall i \notin T \wedge |U| \leq d - q\}. \quad (28)$$

For every tuple $\mathbf{a}_{\bar{G}} \in \mathbb{N}^{\bar{G}}$ with $\|\mathbf{a}_{\bar{G}}\|_1 = q < d$, $i \in T$, and every $U \in \mathcal{U}(T, q)$, define the following matrices, which play the same role as $\mathbf{I}_{\text{city}} + \mathbf{R}^\top \mathbf{R}$ in Section 4.1:

$$\mathbf{B}_{T, q, i} = \sum_{U \in \mathcal{U}(T, q)} \left(\begin{bmatrix} \star & \mathbf{R}_c \\ \mathbf{I}_{c \in U \cap G_i} & \mathbf{R}_c \end{bmatrix}^\top \begin{bmatrix} \star & \mathbf{R}_c \\ \mathbf{I}_{c \in U \cap G_i} & \mathbf{R}_c \end{bmatrix} \right), \quad (29)$$

$$\mathbf{R}_{\mathbf{a}_{\bar{G}}, U} = \bigotimes_{\substack{w \in \bar{G} \\ a_w > 0}} \mathbf{I}_w \otimes \bigotimes_{i \in T} \bigotimes_{c \in U \cap G_i} \star \mathbf{R}_c. \quad (30)$$

The following theorem reparameterizes $J(\theta)$ for PR^d ($d \geq 1$) to become $\bar{J}(\gamma)$. While $\theta = (\theta_{\mathbf{a}})$ is a vector indexed by tuples $\mathbf{a} = \mathbf{a}_V \in \mathbb{N}^V$, the new parameters $\gamma = (\gamma_{\mathbf{b}})$ are indexed by integer tuples $\mathbf{b} = \mathbf{b}_{\bar{S}} \in \mathbb{N}^{\bar{S}}$.

Theorem 4.1. Let the PR^d -model with parameters $\theta = (\theta_{\mathbf{a}})_{\|\mathbf{a}_V\|_1 \leq d}$, and k groups of simple FDs $G_i = \{f_i\} \cup S_i$, $i \in [k]$. Define the reparameterization:

$$\gamma_{\mathbf{b}_{\bar{S}}} = \begin{cases} \theta_{(\mathbf{b}_{\bar{G}}, \mathbf{0}_G)} & \mathbf{b}_F = \mathbf{0}_F \\ \sum_{U \in \mathcal{U}(T, q)} \mathbf{R}_{\mathbf{b}_{\bar{G}}, U}^\top \theta_{(\mathbf{b}_{\bar{G}}, \mathbf{1}_{U \cap G})} & T = \{j \mid j \in F, b_{f_j} = 1\}, \\ & q = \|\mathbf{b}_{\bar{G}}\|_1 \end{cases}.$$

Then, minimizing $J(\theta)$ is equivalent to minimizing the function

$$\bar{J}(\gamma) = \frac{1}{2} \gamma^\top \bar{\Sigma} \gamma - \langle \gamma, \bar{c} \rangle + \frac{\lambda}{2} \Omega(\gamma), \quad (31)$$

where

$$\begin{aligned} \Omega(\gamma) = \sum_{\substack{\|\mathbf{b}_{\bar{S}}\|_1 \leq d \\ \|\mathbf{b}_F\|_1 = 0}} \|\gamma_{\mathbf{b}_{\bar{S}}}\|_2^2 + \sum_{\substack{\|\mathbf{b}_{\bar{G}}\|_1 = q \\ q < d}} \sum_{\substack{T \subseteq [k] \\ 0 < |T| \leq d - q}} \left\langle \left(\bigotimes_{\substack{w \in \bar{G} \\ b_w > 0}} \mathbf{I}_w \otimes \bigotimes_{i \in T} \mathbf{B}_{T, q, i}^{-1} \right) \gamma_{(\mathbf{b}_{\bar{G}}, \mathbf{1}_{F_T|F})}, \gamma_{(\mathbf{b}_{\bar{G}}, \mathbf{1}_{F_T|F})} \right\rangle. \end{aligned}$$

(Recall $\bar{\Sigma}$ and $\bar{c}$ from Definition 2.)

The proof of this theorem (in [4]) is technically involved. $\bar{J}$ is defined above with respect to the FD-reduced pair of functions $\bar{g}, \bar{h}$ and a reduced parameter space of γ . Its gradient is simple to compute, since

$$\frac{1}{2} \frac{\partial \Omega(\gamma)}{\partial \gamma_{\mathbf{b}_{\bar{S}}}} = \begin{cases} \gamma_{\mathbf{b}_{\bar{S}}} & \text{when } \mathbf{b}_F = \mathbf{0}_F, \\ \left(\bigotimes_{\substack{w \in \bar{G} \\ b_w > 0}} \mathbf{I}_w \otimes \bigotimes_{i \in T} \mathbf{B}_{T, q, i}^{-1} \right) \gamma_{(\mathbf{b}_{\bar{G}}, \mathbf{1}_{F_T|F})} & \text{when } T = \{j \mid \substack{j \in F, \\ b_{f_j} = 1\}, q = \|\mathbf{b}_{\bar{G}}\|_1. \end{cases} \quad (32)$$

Moreover, once a minimizer γ of $\bar{J}$ is obtained, we can compute a minimizer θ of J by setting

$$\theta_{a_V} = \begin{cases} \gamma_{a_{\bar{S}}}, & \text{when } \|a_G\|_1 = 0 \\ \left(\bigotimes_{\substack{w \in \bar{G} \\ a_w > 0}} \mathbf{I}_w \otimes \bigotimes_{i \in T} \left[\star_{c \in U \cap G_i} \mathbf{R}_c \right] \mathbf{B}_{T,q,i}^{-1} \right) \gamma_{(a_{\bar{G}}, 1_{F_T|F})}, & \text{when } \|a_G\|_1 > 0, \|a_{\bar{G}}\|_1 = q, \\ & T = \{i \mid \exists c \in G_i, a_c > 0\} \\ & \text{and } U = \{c \mid a_c > 0, c \in G\}. \end{cases} \quad (33)$$

Theorem 4.1 might be a bit difficult to grasp at first glance due to its generality. To give the reader a sense of how the theorem is applied in specific instances, [4] presents two specializations of the theorem for (ridge) linear regression (PR¹), and degree-2 polynomial regression (PR²).

4.4 Factorization machines under FDs

We now turn our attention to FaMa_r².

Theorem 4.2. Consider the FaMa model of degree 2, rank r , parameters $\theta = (\theta_i, (\theta_i^{(\ell)})_{\ell \in [r]})_{i \in V}$ and k groups of simple FDs $G_i = \{f_i\} \cup S_i, i \in [k]$. Let $G = \bigcup_{i \in [k]} G_i$,

$$\beta_{f_i} := \sum_{\ell=1}^r \sum_{\{c,t\} \in \binom{G_i}{2}} \mathbf{R}_c^\top \theta_c^{(\ell)} \circ \mathbf{R}_t^\top \theta_t^{(\ell)}, \quad i \in [k] \quad (34)$$

and the following reparameterization:

$$\gamma_w = \begin{cases} \theta_w & w \notin \bigcup_{i=1}^k G_i \\ \theta_{f_i} + \sum_{c \in S_i} \mathbf{R}_c^\top \theta_c + \beta_{f_i} & w = f_i, i \in [k]. \end{cases}$$

$$\gamma_w^{(\ell)} = \begin{cases} \theta_w^{(\ell)} & w \notin F \\ \theta_{f_i}^{(\ell)} + \sum_{c \in S_i} \mathbf{R}_c^\top \theta_c^{(\ell)} & w = f_i, i \in [k]. \end{cases}$$

Then, minimizing $J(\theta)$ is equivalent to minimizing the function $\bar{J}(\gamma) := \frac{1}{2} \bar{g}(\gamma)^\top \bar{\Sigma} \bar{g}(\gamma) - \langle \bar{g}(\gamma), \bar{c} \rangle + \frac{\lambda}{2} \Omega(\gamma)$, where

$$\Omega(\gamma) := \sum_{w \notin G} \|\gamma_w\|_2^2 + \sum_{i=1}^k \left\langle \mathbf{B}_i^{-1}(\gamma_{f_i} - \beta_{f_i}), (\gamma_{f_i} - \beta_{f_i}) \right\rangle + \sum_{\substack{\ell \in [r] \\ w \notin F}} \|\gamma_w^{(\ell)}\|_2^2 + \sum_{\substack{i \in [k] \\ \ell \in [r]}} \left\| \gamma_{f_i}^{(\ell)} - \sum_{c \in S_i} \mathbf{R}_c^\top \gamma_c^{(\ell)} \right\|_2^2. \quad (35)$$

(Recall $\bar{g}$, $\bar{\Sigma}$ and $\bar{c}$ from Definition 2.)

In order to optimize $\bar{J}$ with respect to γ , the following proposition provides a closed form formulae for the relevant gradient.

Proposition 4.3. The gradient of $\Omega(\gamma)$ defined in (35) can be computed by computing $\delta_i^{(\ell)} = \sum_{c \in S_i} \mathbf{R}_c^\top \gamma_c^{(\ell)}$, and

$$\beta_{f_i} = \sum_{\ell=1}^r \left[\left(\gamma_{f_i}^{(\ell)} - \frac{1}{2} \delta_i^{(\ell)} \right) \circ \delta_i^{(\ell)} - \frac{1}{2} \sum_{t \in S_i} \mathbf{R}_t^\top (\gamma_t^{(\ell)} \circ \gamma_t^{(\ell)}) \right]$$

Then,

$$\frac{1}{2} \frac{\partial \Omega(\gamma)}{\partial \gamma_w} = \begin{cases} \gamma_w, & w \notin G \\ \mathbf{B}_i^{-1}(\gamma_{f_i} - \beta_{f_i}), & w = f_i, i \in [k]. \end{cases} \quad (36)$$

$$\frac{1}{2} \frac{\partial \Omega(\gamma)}{\partial \gamma_w^{(\ell)}} = \begin{cases} \gamma_w^{(\ell)}, & \text{for } w \notin G, \ell \in [r] \\ \gamma_{f_i}^{(\ell)} - \delta_i^{(\ell)} - \frac{1}{2} \delta_i^{(\ell)} \circ \frac{\partial \Omega(\gamma)}{\partial \gamma_{f_i}}, & \text{for } w = f_i, \ell \in [r] \\ \gamma_w^{(\ell)} - \mathbf{R}_w \left[\gamma_{f_i}^{(\ell)} \circ \frac{1}{2} \frac{\partial \Omega(\gamma)}{\partial \gamma_{f_i}} + \frac{1}{2} \frac{\partial \Omega(\gamma)}{\partial \gamma_{f_i}^{(\ell)}} \right], & \text{when } w \in S_i, \ell \in [r]. \end{cases} \quad (37)$$

Suppose that the minimizer γ of $\bar{J}$ has been obtained, then a minimizer θ of J is available in closed form:

$$\theta_w = \begin{cases} \gamma_w & w \in V \setminus G \\ \mathbf{R}_t \mathbf{B}_i^{-1}(\gamma_{f_i} - \beta_{f_i}), & \forall t \in G_i, i \in [k]. \end{cases}$$

$$\theta_w^{(\ell)} = \begin{cases} \gamma_w^{(\ell)}, & \forall w \notin F, \ell \in [r]. \\ \gamma_w^{(\ell)} - \delta_i^{(\ell)}, & w = f_i, \ell \in [r]. \end{cases}$$

This section shows that our technique applies to a non-linear model too. It should be obvious that a similar reparameterization works for FaMa_r^d for any $d \geq 1$. There is some asymmetry in the reparameterization of 1st-order parameters θ_i and 2nd-order parameters $\theta_i^{(\ell)}$ in Theorem 4.2, because we can solve a system of linear equation with matrix inverses, but we don't have closed form solutions for quadratic equations.

4.5 Linear algebra with database queries

To apply the above results, we need to solve several computational primitives. The first primitive is to compute the matrix inverse $\mathbf{B}_{T,q}^{-1}$ and its product with another vector. This task can be done by either explicitly computing the inverse, or computing the Cholesky decomposition of the matrix $\mathbf{B}_{T,q}$. We next explain how both of these tasks can be done using database queries.

Maintaining the matrix inverse with rank-1 updates. Using Sherman-Morrison-Woodbury formula [4, 27], we can incrementally compute the inverse of the matrix $\mathbf{I} + \sum_{c \in G_i} \mathbf{R}_c^\top \mathbf{R}_c$ as follows. Let $S \subset G_i$ be some subset and suppose we have already computed the inverse for $\mathbf{M}_S = \mathbf{I} + \sum_{s \in S} \mathbf{R}_s^\top \mathbf{R}_s$. We now explain how to compute the inverse for $\mathbf{M}_{S \cup \{c\}} = \mathbf{I} + \sum_{s \in S \cup \{c\}} \mathbf{R}_s^\top \mathbf{R}_s$. For concreteness, let the matrix $\mathbf{R}_c$ map city to country. For each country country, let $\mathbf{e}_{\text{country}}$ denote the 01-vector where there is a 1 for each city the country has. For example, $\mathbf{e}_{\text{cuba}} = [1 \ 1 \ 0 \ 0 \ 0]^\top$. Then, $\mathbf{R}_c^\top \mathbf{R}_c = \sum_{\text{country}} \mathbf{e}_{\text{country}} \mathbf{e}_{\text{country}}^\top$. And thus, starting with $\mathbf{M}_S$, we apply the Sherman-Morrison-Woodbury formula for each country, such as:

$$(\mathbf{M} + \mathbf{e}_{\text{cuba}} \mathbf{e}_{\text{cuba}}^\top)^{-1} = \mathbf{M}^{-1} - \frac{\mathbf{M}^{-1} \mathbf{e}_{\text{cuba}} \mathbf{e}_{\text{cuba}}^\top \mathbf{M}^{-1}}{1 + \mathbf{e}_{\text{cuba}}^\top \mathbf{M}^{-1} \mathbf{e}_{\text{cuba}}}. \quad (38)$$

This update can be done with database aggregate queries, because $\mathbf{e}_{\text{cuba}}^\top \mathbf{M}^{-1} \mathbf{e}_{\text{cuba}}$ is a sum of entries (i, j) in $\mathbf{M}^{-1}$ where both i and j are cities in cuba; $\mathbf{v} = \mathbf{M}^{-1} \mathbf{e}_{\text{cuba}}$ is the sum of columns of $\mathbf{M}^{-1}$ corresponding to cuba; and $\mathbf{M}^{-1} \mathbf{e}_{\text{cuba}} \mathbf{e}_{\text{cuba}}^\top \mathbf{M}^{-1}$ is exactly $\mathbf{v} \mathbf{v}^\top$.

Overall, each update (38) can be done in $O(N_{\text{city}}^2)$ -time, for an overall runtime of $O(N_{\text{city}}^2 N_{\text{country}})$. This runtime should be contrasted with Gaussian-elimination-based inverse computation time,

which is $O(N_{\text{city}}^3)$. When the FDs form a chain, the blocks are nested inside one another, and thus each update is even cheaper as we do not have to access all N_{city}^2 entries.

Maintaining a Cholesky decomposition with rank- k update. Maintaining a matrix inverse can be numerically unstable. It would be best to compute a Cholesky decomposition of the matrix, since this strategy is numerically more stable. There are known rank-1 update algorithms [18, 23], using strategies similar to the inverse rank-1 update above. A further common computational primitive is to multiply a tensor product with a vector, such as in $(\mathbf{B}_i^{-1} \otimes \mathbf{B}_j^{-1})\gamma_{f_{ij}}$ (also expressible as aggregate queries, cf. Appendix A.2).

4.6 Discussion

The prevalence of FDs presents fresh new challenges from both computational and statistical viewpoints. On the one hand, a reasonable and well-worn rule of thumb in statistics dictates that one should always eliminate features that are functionally dependent on others, because this helps reduce both computation and model's complexity, which in turn leads to reduced generalization error (as also noted in [36]). On the other hand, the statistical effectiveness of such a rule is difficult to gauge when the nature of dependence goes beyond linearity. In such scenarios, it might be desirable to keep some redundant variables, but only if they help construct simpler forms of regression/classification functions, leading to improved approximation ability for the model class.

It is, however, difficult to know a priori which redundant features lead to simple functions. Therefore, the problem of dimensionality reduction cannot be divorced from the model class under consideration. While this remains unsolved in general, in this work we restricted ourselves to specific classes of learning models, the complexity of which may still be varied through regularization via (non-linear) penalties. Within a regularized parametric model class, we introduced dimensionality reduction techniques (variable elimination and re-parameterization) that may not fundamentally change the model's capacity. The reduction in the number of parameters may still help reduce the variance of parameter estimates, leading to improved generalization error guarantees.

We pursued in this section a more tangible benefit that lies in improved computational efficiency. There is substantial runtime saving brought by the reparameterizations from Theorems 4.1 and 4.2. In the setting without exploiting FDs, the runtime of our approach consists of the precomputation time stated in Proposition 3.4, and the per-iteration time stated in Theorem 3.6; this time might be multiplied by the number of backtracking steps depending on the specific iteration. By exploiting the FDs, the quantities in Propositions 3.4 and 3.6 are computed with respect to the smaller dimensional matrix $\bar{\Sigma}$. Let $q = |V - \bigcup_{i \in [k]} S_i|$, then the dimensionality of $\bar{\Sigma}$ for PR^d is $\Theta(q^d) \times \Theta(q^d)$, much smaller than that of Σ , which is $\Theta(n^d) \times \Theta(n^d)$. (See Example 13.) Last but not least, [4] provides the corresponding version of Corollary 3.2 with respect to $\bar{\Sigma}$.

5 OPEN PROBLEMS

Our in-database learning framework raises open questions on statistics, algorithm design, and optimization. We next sketch a few representative questions.

One research direction is to extend the class of statistical models to train efficiently inside the database beyond those considered in this paper. Our formulation (1) captures a common class of regression models (such as PR and FaMa) and classification models (such as logistic and SVM), which is done by changing the loss function $\mathcal{L}$. It remains open how to extend our formulation to capture latent variable models.

The aggregates defining Σ , $\mathbf{c}$, point evaluation, and gradient computation are “multi-output” queries. They deserve a systematic investigation, from formulation to evaluation and complexity analysis. In practice, one often reserves a fragment of the training data for model validation. It is an interesting question to incorporate this data partitioning requirement into our framework.

Understanding how to adapt further optimization algorithms, such as coordinate descent or stochastic gradient, to our in-database framework is an important research direction. Furthermore, our FD-aware optimization is specific to the ℓ_2 -norm in the penalty term. We would also like to understand the effect of other norms, e.g., ℓ_1 , on model reparameterization under FDs.

Finally, we conjecture that the cost function may be easier to optimize with respect to the reduced set of parameters that are not functionally determined: As redundant variables are eliminated or optimized out, the cost function's Hessian with respect to reduced parameters becomes less ill-conditioned, resulting in faster convergence behavior for gradient-based optimization techniques. The impact of FD-based dimensionality reduction, from both computational and statistical standpoints, have not been extensively studied for learning (nonlinear) models with categorical variables, which are precisely the kind discussed in our framework.

ACKNOWLEDGMENTS

This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 682588. XN is supported in part by grants NSF CAREER DMS-1351362, NSF CNS-1409303 and the Margaret and Herman Sokol Faculty Award.

REFERENCES

- [1] Martin Abadi et al. 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. *CoRR* abs/1603.04467 (2016).
- [2] Serge Abiteboul et al. 2017. Research Directions for Principles of Data Management (Dagstuhl Perspectives Workshop 16151). *CoRR* abs/1701.09007 (2017).
- [3] S. Abiteboul, R. Hull, and V. Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [4] Mahmoud Abo Khamis, Hung Ngo, XuanLong Nguyen, Dan Olteanu, and Maximilian Schleich. 2017. In-Database Learning with Sparse Tensors. *CoRR* abs/1703.04780 (2017).
- [5] Mahmoud Abo Khamis, Hung Ngo, XuanLong Nguyen, Dan Olteanu, and Maximilian Schleich. 2018. AC/DC: In-Database Learning Thunderstruck. *CoRR* abs/1803.07480 (2018).
- [6] Mahmoud Abo Khamis, Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2016. Joins via Geometric Resolutions: Worst Case and Beyond. *ACM Trans. Database Syst.* 41, 4 (2016), 22:1–22:45.
- [7] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. 2015. FAQ: Questions Asked Frequently. *CoRR* abs/1504.04044 (2015).
- [8] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. 2016. FAQ: Questions Asked Frequently. In *PODS*. 13–28.
- [9] Isolde Adler. 2006. *Width functions for hypertree decompositions*. Ph.D. Dissertation, Albert-Ludwigs-Universität Freiburg. 2006.
- [10] Rakesh Agrawal, Heikki Mannila, Ramakrishnan Srikant, Hannu Toivonen, and A. Inkeri Verkamo. 1996. Advances in Knowledge Discovery and Data Mining. Chapter Fast Discovery of Association Rules, 307–328.

- [11] Molham Aref, Balder ten Cate, Todd J. Green, Benny Kimelfeld, Dan Olteanu, Emir Pasalic, Todd L. Veldhuizen, and Geoffrey Washburn. 2015. Design and Implementation of the LogicBlox System. In *SIGMOD*. 1371–1382.
- [12] Albert Aterias, Martin Grohe, and Dániel Marx. 2008. Size Bounds and Query Plans for Relational Joins. In *FOCS*. 739–748.
- [13] Nurzhan Bakibayev, Tomás Kociský, Dan Olteanu, and Jakub Závodný. 2013. Aggregation and Ordering in Factorised Databases. *PVLDB* 6, 14 (2013), 1990–2001.
- [14] Jonathan Barzilai and Jonathan M. Borwein. 1988. Two-point step size gradient methods. *IMA J. Numer. Anal.* 8, 1 (1988), 141–148.
- [15] Matthias Boehm, Shirish Tatikonda, Berthold Reinwald, Prithviraj Sen, Yuanyuan Tian, Douglas Burdick, and Shivakumar Vaithyanathan. 2014. Hybrid Parallelization Strategies for Large-Scale Machine Learning in SystemML. *PVLDB* 7, 7 (2014), 553–564.
- [16] Léon Bottou. 2012. Stochastic Gradient Descent Tricks. In *Neural Networks: Tricks of the Trade (2nd ed)*. 421–436.
- [17] Jean-Francois Boulicaut and Cyrille Masson. 2005. *Data Mining Query Languages*. 715–726.
- [18] Timothy A. Davis and William W. Hager. 2001. Multiple-rank modifications of a sparse Cholesky factorization. *SIAM J. Matrix Anal. Appl.* 22, 4 (2001), 997–1013.
- [19] Tarek Elgamal, Shangyu Luo, Matthias Boehm, Alexandre V. Evfimievski, Shirish Tatikonda, Berthold Reinwald, and Prithviraj Sen. 2017. SPOOF: Sum-Product Optimization and Operator Fusion for Large-Scale Machine Learning. In *CIDR*.
- [20] Rong-En Fan et al. 2008. LIBLINEAR: A Library for Large Linear Classification. *J. Mach. Learn. Res.* 9 (2008), 1871–1874.
- [21] Xixuan Feng, Arun Kumar, Benjamin Recht, and Christopher Ré. 2012. Towards a unified architecture for in-RDBMS analytics. In *SIGMOD*. 325–336.
- [22] Roger Fletcher. 2005. On the Barzilai-Borwein method. In *Optimization and control with applications*. Appl. Optim., Vol. 96. 235–256.
- [23] P. E. Gill, G. H. Golub, W. Murray, and M. A. Saunders. 1974. Methods for modifying matrix factorizations. *Math. Comp.* 28 (1974), 505–535.
- [24] Tom Goldstein, Christoph Studer, and Richard G. Baraniuk. 2014. A Field Guide to Forward-Backward Splitting with a FASTA Implementation. *CoRR abs/1411.3406* (2014).
- [25] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 1999. Hypertree decompositions and tractable queries. In *PODS*. 21–32.
- [26] Martin Grohe and Dániel Marx. 2014. Constraint Solving via Fractional Edge Covers. *ACM Trans. Alg.* 11, 1 (2014), 4:1–4:20.
- [27] William W. Hager. 1989. Updating the inverse of a matrix. *SIAM Rev.* 31, 2 (1989), 221–239.
- [28] David Harris and Sarah Harris. 2012. *Digital Design and Computer Architecture* (2nd ed.).
- [29] T. Hastie, R. Tibshirani, and M. J. Wainwright. 2015. *Statistical Learning with Sparsity: The Lasso and generalizations*. CRC Press.
- [30] Joseph M. Hellerstein et al. 2012. The MADlib Analytics Library or MAD Skills, the SQL. *PVLDB* 5, 12 (2012), 1700–1711.
- [31] Botong Huang, Matthias Boehm, Yuanyuan Tian, Berthold Reinwald, Shirish Tatikonda, and Frederick R. Reiss. 2015. Resource Elasticity for Large-Scale Machine Learning. In *SIGMOD*. 137–152.
- [32] C. G. Khatri and C. Radhakrishna Rao. 1968. Solutions to some functional equations and their applications to characterization of probability distributions. *Sankhyā Ser. A* 30 (1968), 167–180.
- [33] Benny Kimelfeld and Christopher Ré. 2017. A Relational Framework for Classifier Engineering. In *PODS*. 5–20.
- [34] Arun Kumar, Matthias Boehm, and Jun Yang. 2017. Data Management in Machine Learning: Challenges, Techniques, and Systems. In *SIGMOD*. 1717–1722.
- [35] Arun Kumar, Jeffrey F. Naughton, and Jignesh M. Patel. 2015. Learning Generalized Linear Models Over Normalized Data. In *SIGMOD*. 1969–1984.
- [36] Arun Kumar, Jeffrey F. Naughton, Jignesh M. Patel, and Xiaojin Zhu. 2016. To Join or Not to Join?: Thinking Twice about Joins before Feature Selection. In *SIGMOD*. 19–34.
- [37] Xiangrui Meng et al. 2016. MLlib: Machine Learning in Apache Spark. *J. Mach. Learn. Res.* 17, 1 (2016), 1235–1241.
- [38] Dirk Neumann. 2015. Lightning-Fast Deep Learning on Spark Via parallel stochastic gradient updates, www.deeplearning.com. (2015).
- [39] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2012. Worst-case Optimal Join Algorithms. In *PODS*. 37–48.
- [40] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2013. Skew Strikes Back: New Developments in the Theory of Join Algorithms. In *SIGMOD Rec.* 5–16.
- [41] Dan Olteanu and Jakub Závodný. 2015. Size Bounds for Factorised Representations of Query Results. *ACM Trans. Database Syst.* 40, 1 (2015), 2:1–2:44.
- [42] Jian Pei, Jiawei Han, and Laks VS Lakshmanan. 2001. Mining frequent itemsets with convertible constraints. In *ICDE*. 433–442.
- [43] K. B. Petersen and M. S. Pedersen. 2012. The Matrix Cookbook. (nov 2012). <http://www2.imm.dtu.dk/pubdb/p.php?3274> Version 20121115.
- [44] Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2017. Data Management Challenges in Production Machine Learning. In *SIGMOD*.

1723–1726.

- [45] Chengjie Qin and Florin Rusu. 2015. Speculative Approximations for Terascale Distributed Gradient Descent Optimization. In *DanaC*. 1:1–1:10.
- [46] Steffen Rendle. 2012. Factorization Machines with libFM. *ACM Trans. Intell. Syst. Technol.* 3, 3 (2012), 57:1–57:22.
- [47] Steffen Rendle. 2013. Scaling Factorization Machines to Relational Data. *PVLDB* 6, 5 (2013), 337–348.
- [48] Maximilian Schleich, Dan Olteanu, and Radu Ciucanu. 2016. Learning Linear Regression Models over Factorized Joins. In *SIGMOD*. 3–18.
- [49] Todd L. Veldhuizen. 2014. Triejoin: A Simple, Worst-Case Optimal Join Algorithm. In *ICDT*. 96–106.
- [50] Matei Zaharia et al. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *NSDI*. 15–28.

A BACKGROUND

A.1 Tensor product, Kronecker product, and Khatri-Rao product

We discuss some identities for tensors. We use $\otimes$ to denote the *tensor product*. When taking the tensor product of two matrices, this is called the *Kronecker product*, which is *not* the same as the outer product for matrices, even though the two are isomorphic maps. If $\mathbf{A} = (a_{ij})$ is an $m \times n$ matrix and $\mathbf{B} = (b_{k\ell})$ is a $p \times q$ matrix, then the tensor product $\mathbf{A} \otimes \mathbf{B}$ is an $mp \times nq$ matrix whose $((i, k), (j, \ell))$ entry is $a_{ij}b_{k\ell}$. In particular, if $\mathbf{x} = (x_i)_{i \in [m]}$ is an m -dimensional vector and $\mathbf{y} = (y_j)_{j \in [p]}$ is a p -dimensional vector, then $\mathbf{x} \otimes \mathbf{y}$ is an mp -dimensional vector whose (i, j) entry is $x_i y_j$; this is *not* an $m \times p$ matrix as in the case of the outer product. This layout is the correct layout from the definition of the tensor (Kronecker) product. If $\mathbf{A}$ is a matrix, then $\mathbf{A}^{\otimes k}$ denotes the tensor power $\underbrace{\mathbf{A} \otimes \cdots \otimes \mathbf{A}}_{k \text{ times}}$.

Definition 3 (Tensor product). Let $\mathbf{A}$ be a tensor of order r , i.e., a function $\psi_A(X_1, \dots, X_r)$, and $\mathbf{B}$ be a tensor of order s , i.e., a function $\psi_B(Y_1, \dots, Y_s)$, then the tensor product $\mathbf{A} \otimes \mathbf{B}$ is the multilinear function

$$\psi(X_1, \dots, X_r, Y_1, \dots, Y_s) = \psi_A(X_1, \dots, X_r) \psi_B(Y_1, \dots, Y_s).$$

(A matrix is a tensor of order 2.)

Definition 4 (Khatri-Rao product). Let $\mathbf{A}$ and $\mathbf{B}$ be two matrices each with n columns. We use $\mathbf{A} \star \mathbf{B}$ to denote the matrix with n columns, where the j th column of $\mathbf{A} \star \mathbf{B}$ is the tensor product of the j th column of $\mathbf{A}$ with the j th columns of $\mathbf{B}$. The operator $\star$ is a (special case of) the *Khatri-Rao product* [32], where we partition the input matrices into blocks of one column each. More elaborately, if $\mathbf{A}$ has columns $\mathbf{a}_1, \dots, \mathbf{a}_n$, and $\mathbf{B}$ has columns $\mathbf{b}_1, \dots, \mathbf{b}_n$, then

$$\mathbf{A} \star \mathbf{B} = [\mathbf{a}_1 \otimes \mathbf{b}_1 \quad \mathbf{a}_2 \otimes \mathbf{b}_2 \quad \cdots \quad \mathbf{a}_n \otimes \mathbf{b}_n].$$

(Note $\mathbf{A}$ and $\mathbf{B}$ do not need to have the same number of rows.)

Definition 5 (Hadamard product). Let $\mathbf{A} = (a_{ij})$ and $\mathbf{B} = (b_{ij})$ be two $m \times n$ matrices, then the Hadamard product $\mathbf{A} \circ \mathbf{B}$ is an $m \times n$ matrix, where each i, j element is given by $(\mathbf{A} \circ \mathbf{B})_{ij} = a_{ij}b_{ij}$.

Proposition A.1. *The following equalities hold, assuming the dimensionalities match up correctly:*

$$(AB \otimes CD) = (A \otimes C)(B \otimes D) \quad (39)$$

$$(A \otimes B)^T = (A^T \otimes B^T) \quad (40)$$

$$\langle \mathbf{x}, B\mathbf{y} \rangle = \langle B^T \mathbf{x}, \mathbf{y} \rangle \quad (41)$$

$$(A \otimes B)^{-1} = (A^{-1} \otimes B^{-1}) \text{ if both are square matrices} \quad (42)$$

$$\langle A \otimes B, R\mathbf{x} \otimes S\mathbf{y} \rangle = \langle R^T A \otimes S^T B, \mathbf{x} \otimes \mathbf{y} \rangle. \quad (43)$$

If $\mathbf{x}$ is a standard n -dimensional unit vector, A and B are two matrices with n columns each, and $\mathbf{a}$ and $\mathbf{b}$ are two n -dimensional vectors, then

$$(A \otimes B)(\mathbf{x} \otimes \mathbf{x}) = (A \star B)\mathbf{x} \quad (44)$$

$$\langle \mathbf{a} \otimes \mathbf{b}, \mathbf{x} \otimes \mathbf{x} \rangle = \langle \mathbf{a} \circ \mathbf{b}, \mathbf{x} \rangle. \quad (45)$$

Let $\mathbf{x}$ be a standard n -dimensional unit vector, $A_1, \dots, A_k$ be k matrices with n columns each. Then,

$$\left(\bigotimes_{i=1}^k A_i \right) (\mathbf{x}^{\otimes k}) = \left(\bigstar_{i=1}^k A_i \right) \mathbf{x}. \quad (46)$$

The first five identities are used in our dimension reduction techniques that exploit functional dependencies, while (44), (45), and (46) are instrumental in achieving computational reduction in our handling of categorical features. See [4, 43] for a proof.

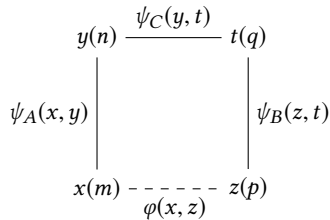
A.2 Tensor computation, FAQ-expression, and the InsideOut algorithm

Quite often we need to compute a product of the form $(A \otimes B)C$, where A , B , and C are tensors, provided that their dimensionalities match up. For example, suppose A is an $m \times n$ matrix, B a $p \times q$ matrix, and C a $nq \times 1$ matrix (i.e. a vector). The result is a $mp \times 1$ tensor. The brute-force way of computing $(A \otimes B)C$ is to compute $A \otimes B$ first, taking $\Theta(mnpq)$ -time, and then multiply the result with C , for an overall runtime of $\Theta(mnpq)$. The brute-force algorithm is a horribly inefficient algorithm.

An alternative way to compute $(A \otimes B)C$ is to view it in a sum-product form, i.e., as an FAQ-expression [8]: A is seen as a function $\psi_A(x, y)$, B as a function $\psi_B(z, t)$, and C as a function $\psi_C(y, t)$. We would then want to compute the function

$$\varphi(x, z) = \sum_y \sum_t \psi_A(x, y) \psi_B(z, t) \psi_C(y, t). \quad (47)$$

This is a 4-cycle FAQ-query:



To compute it, we can pick between the following two strategies:

- eliminate t first (i.e. compute $\varphi_1(y, z) := \sum_t \psi_B(z, t) \psi_C(y, t)$ with a runtime of $O(npq)$), and then eliminate y (i.e. compute $\varphi(x, y) = \sum_y \varphi_1(y, z) \psi_A(x, y)$ with $O(mnp)$ -time. The overall runtime is thus $O(np(m + q))$.
- or the symmetric strategy of eliminating y first, and then t for an overall runtime of $O(mq(n + p))$.

This is not surprising, since the problem is matrix chain multiplication. Using the InsideOut algorithm for FAQ expressions, we want to pick the best tree decomposition and then compute a variable elimination order out of it [8]. A special case of the above is when $B = I$, the identity matrix. In that case, $\psi_B(z, t)$ is the same as the atom $z = t$, and thus it serves as a change of variables:

$$\varphi(x, z) = \sum_y \sum_t \psi_A(x, y) \psi_B(z, t) \psi_C(y, t) = \sum_y \psi_A(x, y) \psi_C(y, z).$$

In other words, we only have to marginalize out one variable instead of two. This situation arises, for example, in Eq. (32) and Eq. (33).

A.3 FAQ-width

A.3.1 Fractional edge cover number and output size bounds. We consider a conjunctive query Q over a relational database instance I . We use N to denote the size of the largest input relation in Q . We also use $Q(I)$ to denote the output and $|Q(I)|$ to denote its size. We use the query Q and its hypergraph $\mathcal{H}$ interchangeably.

Definition 6 (Fractional edge cover number ρ^*). Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be a hypergraph. Let $B \subseteq \mathcal{V}$ be any subset of vertices. A *fractional edge cover* of B using edges in $\mathcal{H}$ is a feasible solution $\lambda = (\lambda_S)_{S \in \mathcal{E}}$ to the following linear program:

$$\begin{aligned} \min \quad & \sum_{S \in \mathcal{E}} \lambda_S \\ \text{s.t.} \quad & \sum_{S: v \in S} \lambda_S \geq 1, \quad \forall v \in B \\ & \lambda_S \geq 0, \quad \forall S \in \mathcal{E}. \end{aligned}$$

The optimal objective value of the above linear program is called the *fractional edge cover number* of B in $\mathcal{H}$ and is denoted by $\rho_{\mathcal{H}}^*(B)$. When $\mathcal{H}$ is clear from the context, we drop the subscript $\mathcal{H}$ and use $\rho^*(B)$. Given a conjunctive query Q , the fractional edge cover number of Q is $\rho_{\mathcal{H}}^*(\mathcal{V})$ where $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ is the hypergraph of Q .

Theorem A.2 (AGM-bound [12, 26]). *Given a full conjunctive query Q over a relational database instance I , the output size is bounded by*

$$|Q(I)| \leq N^{\rho^*},$$

where ρ^* is the fractional edge cover number of Q .

Theorem A.3 (AGM-bound is tight [12, 26]). *Given a full conjunctive query Q and a non-negative number N , there exists a database instance I whose relation sizes are upper-bounded by N and satisfies*

$$|Q(I)| = \Theta(N^{\rho^*}).$$

Worst-case optimal join algorithms [6, 39, 40, 49] can be used to answer any full conjunctive query Q in time

$$O(|\mathcal{V}| \cdot |\mathcal{E}| \cdot N^{\rho^*} \cdot \log N). \quad (48)$$

A.3.2 Tree decompositions, acyclicity, and width parameters.

Definition 7 (Tree decomposition). Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be a hypergraph. A *tree decomposition* of $\mathcal{H}$ is a pair (T, χ) where $T = (V(T), E(T))$ is a tree and $\chi : V(T) \rightarrow 2^{\mathcal{V}}$ assigns to each node of T a subset of vertices of $\mathcal{H}$. The sets $\chi(t)$, $t \in V(T)$, are called the *bags* of the tree decomposition. There are two properties the bags must satisfy

- (a) For any hyperedge $F \in \mathcal{E}$, there is a bag $\chi(t)$, $t \in V(T)$, such that $F \subseteq \chi(t)$.
- (b) For any vertex $v \in \mathcal{V}$, the set $\{t \mid t \in V(T), v \in \chi(t)\}$ is not empty and forms a connected subtree of T .

Definition 8 (acyclicity). A hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ is *acyclic* iff there exists a tree decomposition (T, χ) in which every bag $\chi(t)$ is a hyperedge of $\mathcal{H}$.

When $\mathcal{H}$ represents a join query, the tree T in the above definition is also called the *join tree* of the query. A query is acyclic if and only if its hypergraph is acyclic. For non-acyclic queries, we need a measure of how “close” a query is to being acyclic.

We use the following *width* notions of a query.

Definition 9 (g -width of a hypergraph: a generic width notion [9]). Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, and $g : 2^{\mathcal{V}} \rightarrow \mathbb{R}^+$ be a function that assigns a non-negative real number to each subset of $\mathcal{V}$. The g -width of a tree decomposition (T, χ) of $\mathcal{H}$ is $\max_{t \in V(T)} g(\chi(t))$. The g -width of $\mathcal{H}$ is the *minimum* g -width over all tree decompositions of $\mathcal{H}$. (Note that the g -width of a hypergraph is a *Minimax* function.)

Definition 10 (Treewidth and fractional hypertree width are special cases of g -width). Let s be the following function: $s(B) = |B| - 1$, $\forall B \subseteq \mathcal{V}$. Then the *treewidth* of a hypergraph $\mathcal{H}$, denoted by $\text{tw}(\mathcal{H})$, is exactly its s -width, and the *fractional hypertree width* of a hypergraph $\mathcal{H}$, denoted by $\text{fhtw}(\mathcal{H})$, is the ρ^* -width of $\mathcal{H}$.

From the above definitions, $\text{fhtw}(\mathcal{H}) \geq 1$ for any hypergraph $\mathcal{H}$. Moreover, $\text{fhtw}(\mathcal{H}) = 1$ if and only if $\mathcal{H}$ is acyclic.

A.3.3 Vertex/variable orderings and their equivalence to tree decompositions. Besides tree decompositions, there is another way to define acyclicity and width notions of a hypergraph, which is *orderings* of the hypergraph vertices. Just like we refer to queries and hypergraphs interchangeably, we also refer to query variables and hypergraph vertices interchangeably.

Let n denote the number of vertices of the given hypergraph $\mathcal{H}$.

Definition 11 (Vertex ordering of a hypergraph). A *vertex ordering* of a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ is a listing $\sigma = (v_1, \dots, v_n)$ of all vertices in $\mathcal{V}$.

Definition 12 (Elimination sets U_j^σ of a vertex ordering σ). Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and a vertex ordering $\sigma = (v_1, \dots, v_n)$, we define sets $U_1^\sigma, \dots, U_n^\sigma \subseteq \mathcal{V}$, called the *elimination sets* of σ , as follows: Let $\partial(v_n)$ be the set of hyperedges of $\mathcal{H}$ that contain v_n . We define U_n^σ to be the union of all hyperedges in $\partial(v_n)$:

$$U_n^\sigma := \bigcup_{S \in \partial(v_n)} S.$$

If $n = 1$, then we are done. Otherwise, we remove vertex v_n and all hyperedges in $\partial(v_n)$ from $\mathcal{H}$ and add back to $\mathcal{H}$ a new hyperedge

$U_n^\sigma - \{v_n\}$, thus turning $\mathcal{H}$ into a hypergraph with $n - 1$ vertices:

$$\begin{aligned} \mathcal{V} &\leftarrow \mathcal{V} - \{v_n\}, \\ \mathcal{E} &\leftarrow (\mathcal{E} - \partial(v_n)) \cup \{U_n^\sigma - \{v_n\}\}. \end{aligned}$$

The remaining elimination sets $U_1^\sigma, \dots, U_{n-1}^\sigma$ are defined inductively to be the elimination sets of the resulting hypergraph (whose vertices are now $\{v_1, \dots, v_{n-1}\}$). When σ is clear from the context, we drop the superscript σ and use $U_1, \dots, U_n$.

Proposition A.4 (Every vertex ordering has an “equivalent” tree decomposition [7]). *Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, for every vertex ordering σ , there is a tree decomposition (T, χ) whose bags $\chi(t)$ are the elimination sets U_j^σ of σ .*

By applying the GYO elimination procedure [3] on the bags of a tree decomposition, we can obtain an “equivalent” vertex ordering:

Proposition A.5 (Every tree decomposition has an “equivalent” vertex ordering [7]). *Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, for every tree decomposition (T, χ) , there is a vertex ordering σ such that every elimination set U_j^σ of σ is contained in some bag $\chi(t)$ of the tree decomposition (T, χ) .*

A.3.4 FAQ-width of an FAQ query. Just like a conjunctive query, an FAQ query has a query hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$. But unlike conjunctive queries, an FAQ query also specifies an order of its variables, which is the order in which we aggregate over those variables in the given FAQ-expression. (For example, in expression (47), we sum over t first, then over y , and we keep z and x as free variables. Hence, the FAQ query in (47) specifies the variable order $\sigma = (x, z, y, t)$.) Such a variable order for the query can also be interpreted as a vertex order σ for the query’s hypergraph.

As exemplified in Section A.2, the InsideOut algorithm for answering FAQ queries is based on *variable elimination*. To eliminate variable/vertex v_n , we have to solve a sub-problem consisting of a smaller FAQ query over the variables in the elimination set U_n^σ . This smaller query can be solved by an algorithm that is based on worst-case optimal join algorithms [6, 39, 40, 49]. From (48), this takes time ⁵

$$O(|\mathcal{V}| \cdot |\mathcal{E}| \cdot N^{\rho_{\mathcal{H}}^*(U_n^\sigma)} \cdot \log N). \quad (49)$$

After eliminating v_n , the remaining variables $v_{n-1}, v_{n-2}, \dots, v_1$ can be eliminated similarly. This variable elimination algorithm motivates the following width notion.

Definition 13 (FAQ-width of a given variable ordering σ). Given an FAQ query φ with a variable ordering σ , we define the FAQ-width of σ , denoted by $\text{faqw}(\sigma)$, to be

$$\text{faqw}(\sigma) := \max_{j \in [n]} \left\{ \rho_{\mathcal{H}}^*(U_j^\sigma) \right\}. \quad (50)$$

By the above definition, the FAQ-width of a variable ordering σ is the same as the fractional hypertree width of the “equivalent” tree decomposition that is referred to in Proposition A.4.

Theorem A.6 (Runtime of InsideOut [8]). *Given an FAQ-query φ with a variable order σ , the InsideOut algorithm answers φ in time*

$$O\left(|\mathcal{V}|^2 \cdot |\mathcal{E}| \cdot \left(N^{\text{faqw}(\sigma)} + |\varphi|\right) \cdot \log N\right), \quad (51)$$

⁵To achieve this runtime, we need some additional ideas that are beyond the scope of this very brief introduction to FAQ. See [8] for more details.

where $|\varphi|$ is the output size in the listing representation.

Let φ be an FAQ query with variable ordering σ . In many cases, there might be a different variable ordering σ' such that if we were to permute the aggregates of φ in the order of σ' instead of σ , we would obtain an FAQ-query φ' that is “semantically-equivalent” to φ (i.e. that always returns the same answer as φ no matter what the input is). If this is the case, then we can run InsideOut on φ using the ordering σ' instead of σ , which can lead to a better runtime if $\text{faqw}(\sigma')$ happens to be smaller than $\text{faqw}(\sigma)$. We use $\text{EVO}(\varphi)$ to denote the set of all such “equivalent” orderings σ' . (For a formal definition, see [8].) Therefore, it is best to consider all orderings σ' in $\text{EVO}(\varphi)$, pick the one with the smallest $\text{faqw}(\sigma')$, and use it in the InsideOut algorithm. This motivates the following definition.

Definition 14 (FAQ-width of an FAQ query). The FAQ-width of an FAQ query φ , denoted by $\text{faqw}(\varphi)$, is the minimum one over all orderings σ' in $\text{EVO}(\varphi)$, i.e.

$$\text{faqw}(\varphi) := \min \{ \text{faqw}(\sigma') \mid \sigma' \in \text{EVO}(\varphi) \}. \quad (52)$$

Characterizing $\text{EVO}(\varphi)$ for an arbitrary given FAQ-query φ is a technically involved problem (see [8] for hardness background and a general solution). However, the FAQ queries that we need for our machine learning tasks are of a special form that makes the problem easier. In particular, as discussed in Section A.2, there is only one type of aggregate operators that we use in such queries: This is the summation operator $\sum$. We refer to those special FAQ queries as FAQ-SS queries [8]. Our FAQ-SS queries in this work have only two types of variables:

- Variables that we are summing over, e.g. y and t in (47).
- Free variables (i.e. group-by variables), e.g. x and z in (47).

Given an FAQ-SS query φ , $\text{EVO}(\varphi)$ contains every ordering σ' that lists all free variables *before* the non-free variables. For example, for the FAQ-SS query $\varphi(x, z)$ in (47), $\text{EVO}(\varphi(x, z))$ contains all permutations of $\{x, y, z, t\}$ where $\{x, z\}$ come before $\{y, t\}$.

Proposition A.7. For any FAQ-SS query φ , let $\mathcal{H}$ be its hypergraph. If φ has no free variable, then $\text{faqw}(\varphi) = \text{fhtw}(\mathcal{H})$. If φ has $f \geq 1$ free variables, then $\text{faqw}(\varphi) \leq \text{fhtw}(\mathcal{H}) + f - 1$.

See [4, 7] for a proof of the above proposition.

B MISSING DETAILS FROM SECTION 3

PROOF OF THEOREM 3.1. We start with point evaluation:

$$\begin{aligned} & \frac{1}{2|D|} \sum_{(x, y) \in D} (\langle g(\theta), h(x) \rangle - y)^2 \\ &= \frac{1}{2|D|} \sum_{(x, y) \in D} (\langle g(\theta), h(x) \rangle^2 - 2y \langle g(\theta), h(x) \rangle + y^2) \\ &= \frac{1}{2|D|} \sum_{(x, y) \in D} g(\theta)^\top h(x) h(x)^\top g(\theta) \\ &\quad - \left\langle g(\theta), \frac{1}{|D|} \sum_{(x, y) \in D} y h(x) \right\rangle + \frac{1}{2|D|} \sum_{(x, y) \in D} y^2 \end{aligned}$$

$$\begin{aligned} &= \frac{1}{2} g(\theta)^\top \left(\frac{1}{|D|} \sum_{(x, y) \in D} h(x) h(x)^\top \right) g(\theta) - \langle g(\theta), c \rangle + \frac{SY}{2} \\ &= \frac{1}{2} g(\theta)^\top \Sigma g(\theta) - \langle g(\theta), c \rangle + \frac{SY}{2}. \end{aligned}$$

The gradient formula follows from (11) and the chain rule. $\square$

PROOF OF PROPOSITION 3.4. For any event E , let δ_E denote the Kronecker delta, i.e. $\delta_E = 1$ if E holds, and $\delta_E = 0$ otherwise. Recall that the input query Q has hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, and there is an input relation R_F for every hyperedge $F \in \mathcal{E}$. We can write σ_{ij} in the tensor form as shown in Eq. (18).

Let $C_{ij} := C_i \cup C_j$ and $V_{ij} = V_i \cup V_j$. Plugging in the definition of h_i and h_j from (4), we have

$$\sigma_{ij} = \frac{1}{|D|} \sum_{(x, y) \in D} \prod_{f \in V_{ij} - C_{ij}} x_f^{a_i(f) + a_j(f)} \cdot \bigotimes_{f_i \in C_i} x_{f_i} \otimes \bigotimes_{f_j \in C_j} x_{f_j}.$$

As illustrated in Example 12, the tensor $\bigotimes_{f_i \in C_i} x_{f_i} \otimes \bigotimes_{f_j \in C_j} x_{f_j}$ is very sparse. For a fixed tuple x , in fact, the tensor has only *one* 1 entry, corresponding to the combination of values of the attributes in C_{ij} . Hence, σ_{ij} is a function of the variables C_{ij} . In the FAQ-framework, the query representing σ_{ij} can be expressed as a Sum-Product queries with free (i.e., group-by) variables C_{ij} , defined by:

$$\varphi(C_{ij}) = \frac{1}{|D|} \sum_{x_{f'}: f' \in \mathcal{V} - C_{ij}} \prod_{f \in V_{ij} - C_{ij}} x_f^{a_i(f) + a_j(f)} \cdot \prod_{F \in \mathcal{E}} \delta_{\pi_F(x) \in R_F}. \quad (53)$$

Similarly, the tensor c_j can be sparsely represented by an aggregate query with group-by attributes C_j , which is expressed as the Sum-Product query

$$\varphi(C_j) = \frac{1}{|D|} \sum_{x_{f'}: f' \in \mathcal{V} - C_j} y \cdot \prod_{f \in V_j - C_j} x_f^{a_j(f)} \cdot \prod_{F \in \mathcal{E}} \delta_{\pi_F(x) \in R_F}. \quad (54)$$

The overall runtimes for computing the above FAQ-queries follow from applying the InsideOut algorithm and Theorem A.6 [8]. $\square$

PROOF OF PROPOSITION 3.5. The fact that $\text{faqw}(i, j) \leq \text{fhtw} + c - 1$ follows from Proposition A.7. Since σ_{ij} is a tensor of order at most c , and each attribute's active domain has size at most N , it follows that $|\sigma_{ij}| \leq N^c$. Also, $|\sigma_{ij}| \leq |D|$ since the support of the tensor σ_{ij} cannot be more than the output size.

Fix a query Q with $\rho^* > \text{fhtw} + c - 1 \geq c$. Consider a database instance I for which $|D|$ (the output size of Q) is $\Theta(N^{\rho^*})$. The existence of such database instances is guaranteed by Theorem A.3. Then, (20) follows trivially. $\square$

PROOF OF PROPOSITION 3.6. We first analyze the time it takes to compute expression (11), which is dominated by the quadratic form $g(\theta)^\top \Sigma g(\theta)$. To compute this quadratic form, for every pair $i, j \in [m]$ we need to compute $g_i(\theta)^\top \sigma_{ij} g_j(\theta)$. This product is broken up into a sum of $t_i t_j$ terms when we expand g_i and g_j out. Each of those terms is computed in time $O(d_i d_j |\sigma_{ij}|)$. The runtime for computing (12) is analyzed similarly. $\square$